PENNYMAC · SERVICING SYSTEMS OF ENGAGEMENT

Loan Boarding Platform
Complete Product & Capabilities Guide

How PennyMac onboards a vendor, sets up a feed, picks up files, boards loans into SSE, and resolves exceptions — end to end. Powered by LDS IntelliMap.

Document: LBP-Platform-Guide
Generated: 2026-11-04
Audience: Product, operations, SME, compliance, platform, executives
Start here: §1 Product · Appendix A personas · Appendix B LLD flows

Contents

1. Product description — what LBP is and why it exists
2. The whole process at a glance
3. Phase A — Onboard the vendor
4. Phase B — Setup the feed & publish
5. Phase C — How a file is picked up
6. Phase D — Boarding execution
7. Phase E — Exceptions, outcomes & notifications
8. Who does what — personas
9. Full capability matrix
10. Architecture (control plane vs data plane)
11. Signed configuration & publish events
12. Glue jobs & orchestration
13. Delivery scheduler & vendor pull
14. Vendors, feeds, incremental loads & sequencing
15. Exception tiers and change-file rules
16. Console tour (every screen)
17. Security, compliance & production readiness
18. Operator runbook
19. Web app UX patterns
20. Verified evidence & go-live checklist
21. Glossary
22. Appendix A — Personas in depth
23. Appendix B — LLD: onboarding, daily execution & monitoring

Reading order: product story (§1–7) → people (§8–9) → how it is built (§10–12) → domain depth (§13–15) → screens (§16) → ops (§17–21) → Appendix A (personas) → Appendix B (LLD flows).

1. Product description — what LBP is and why it exists

The problem

PennyMac boards loans from many counterparties (correspondents, bulk sellers, sub-servicers, retail channels). Each sends files in their own layout, on their own schedule, sometimes as full tapes and sometimes as change or snapshot files. Those loans must land correctly in the Servicing System of Engagement (SSE), under regulated rules, with a clear audit trail — without Ops hand-editing rows in production.

What the product is

The **Loan Boarding Platform (LBP)** is the product that:

1. **Onboards** a vendor and their feed(s) into the console.
2. **Sets up** mapping, rules, and delivery behaviour — then **publishes** a signed live configuration.
3. **Picks up** every arriving file (push to S3, scheduled pull, or Ops upload).
4. **Boards** loans through Glue: map → validate → triage → load to SSE, with sequencing and checkpoints.
5. **Routes** bad or risky rows to people by exception tier (auto → reviewer → checker → compliance).
6. **Governs** change with maker-checker, hash-chained audit, and role-based access.

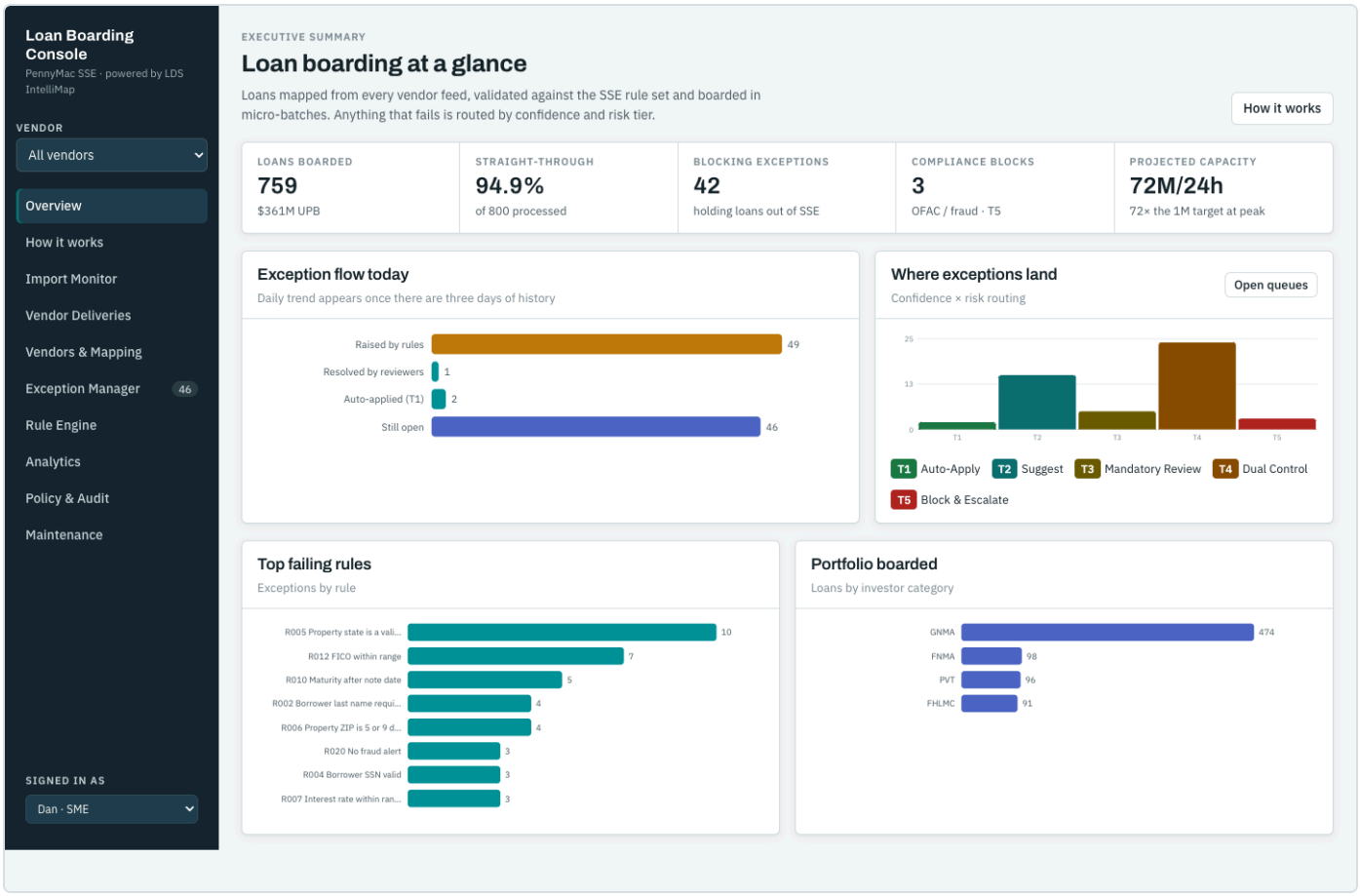
What users see

The **Loan Boarding Console** (React web app) is the day-to-day product surface: Overview, How it works, Import Monitor, Vendor Deliveries, Vendors & Mapping, Exception Manager, Rule Engine, Analytics, Policy & Audit. SME and Platform Admin also get **Maintenance** (read-only Glue / orchestration source).

Behind the console, the **control plane** (API + Lambdas + Step Functions) never transforms loan data. The **data plane** (AWS Glue jobs) does all parsing, mapping, validation and loading — and only from a **signed, published** feed configuration. Drafts never run.

Outcomes the product is judged on

Outcome	How the product delivers it
Straight-through boarding	Published mapping + rules; micro-batch load; reconciliation
Controlled exceptions	Tiers T1–T5; dual control on protected fields; Compliance for OFAC/fraud
Vendor correctness	Per-feed sequencing; delivery windows; late/missed alerts; pull connectors
Change without chaos	Incremental/snapshot load types; change log; optimistic concurrency
Auditability	Maker-checker; signed configs; hash-chained audit; CloudTrail alignment



Overview — the product's executive view (Dan · SME)

2. The whole process at a glance

Everything in LBP is one of two journeys. **Setup** happens once per feed (and again when mapping/rules/delivery change). **Execution** happens for every file.

PHASE A — ONBOARD

Agree counterpart → create **Vendor** → create one or more **Feeds** → set lifecycle (active / suspended)

↓

PHASE B — SETUP

Map columns → attach rules → delivery profile → **SME publish signed config** → optional pull schedule

↓

PHASE C — FILE PICKUP

Three doors into the same pipe: **S3 push** · **scheduled / pull-now** · **Ops upload**

↓

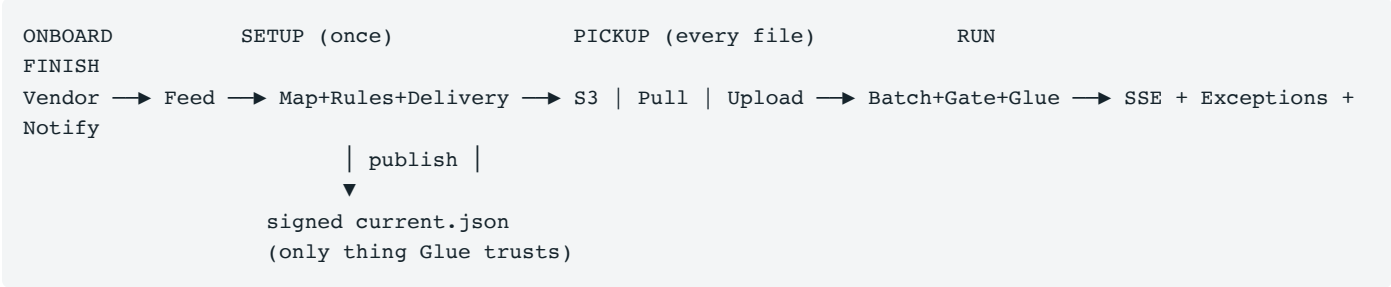
PHASE D — BOARDING RUN

Register batch → sequence gate → route engine → Glue board → checkpoint → reconcile

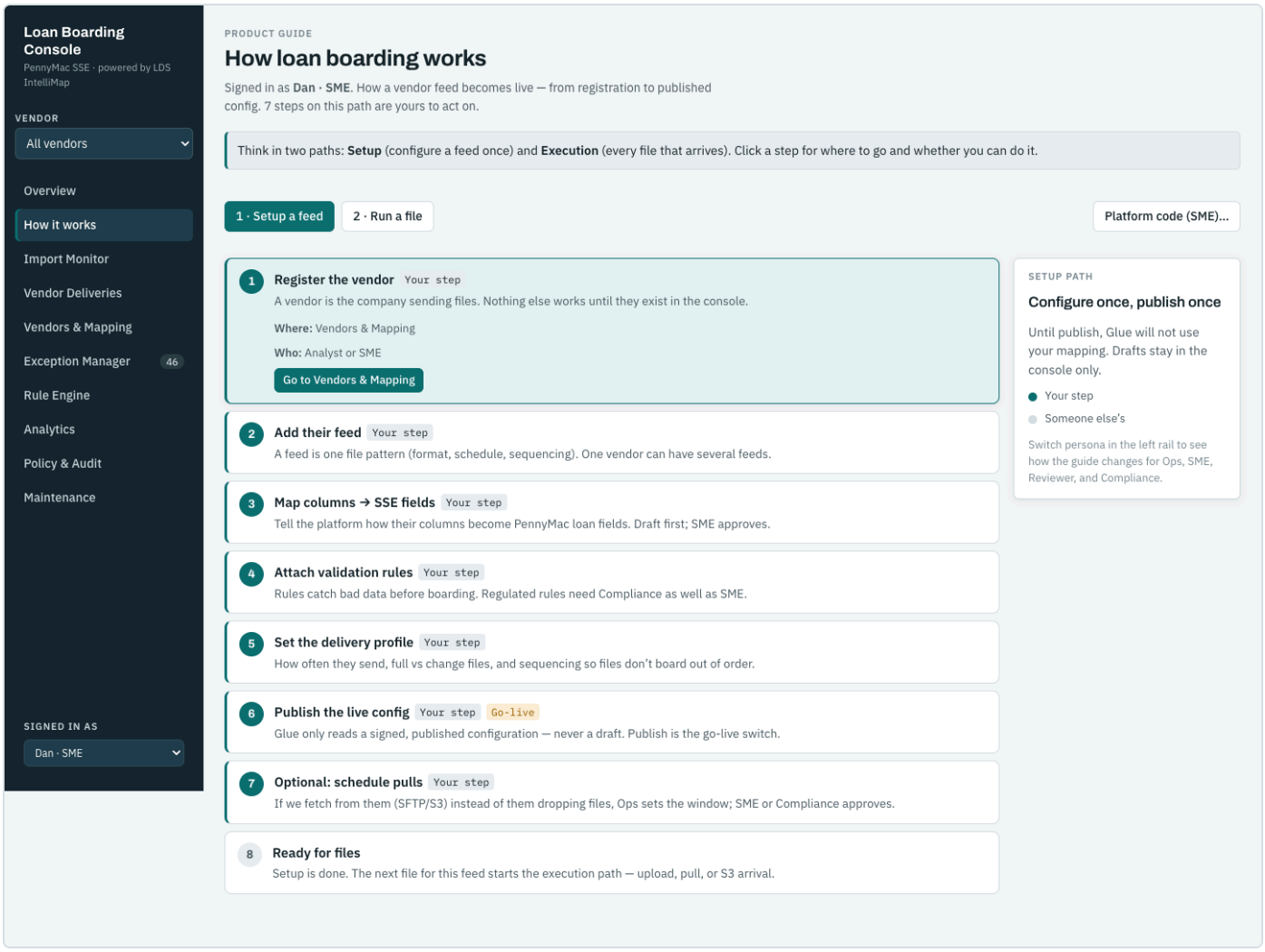
↓

PHASE E — FINISH

Straight-through loans in SSE · exceptions in queues · SNS / Import Monitor status · release waiting files



Read this first. Sections 3–7 walk each phase in product language. Later sections are depth (architecture, CAPS, screens, ops).



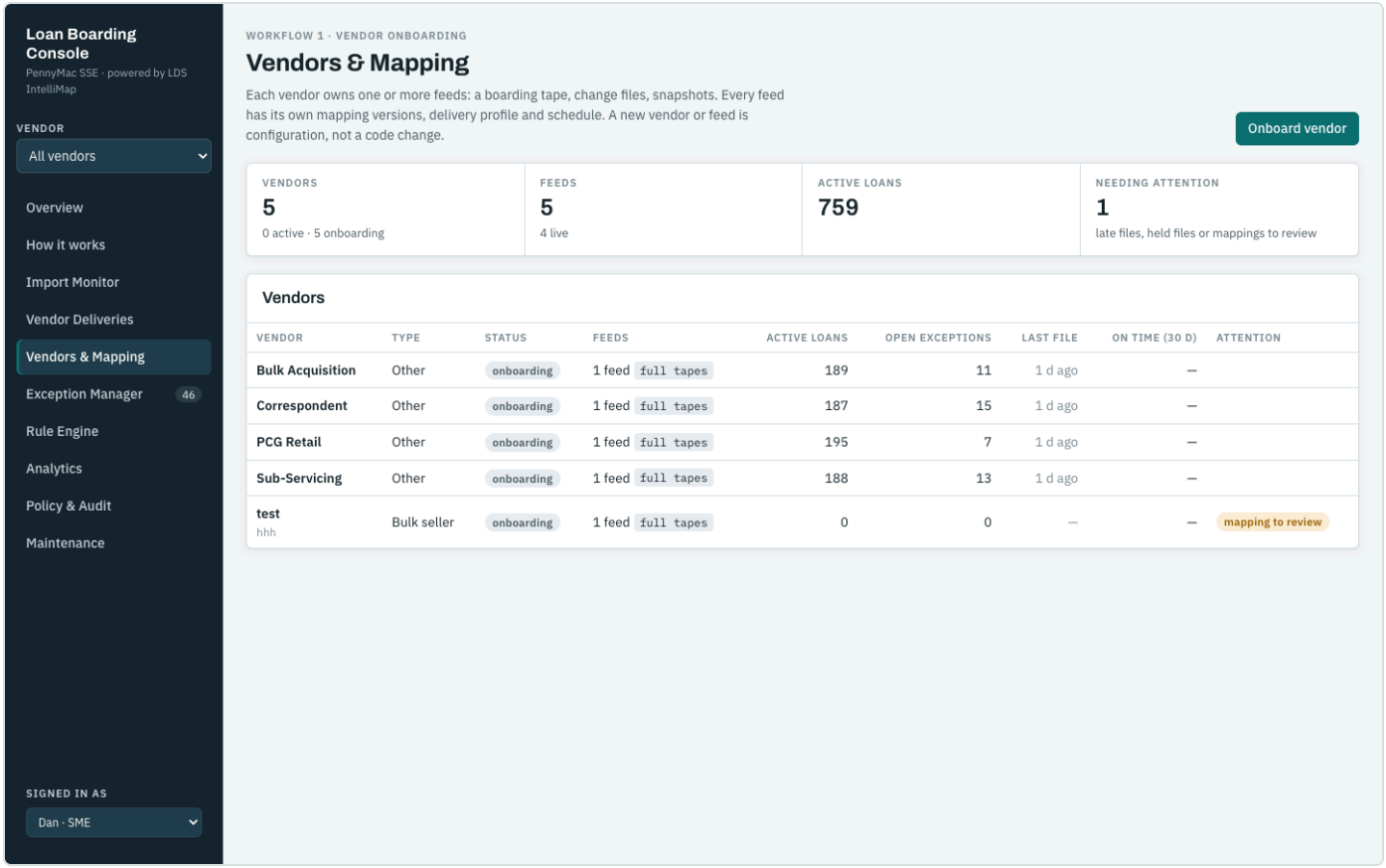
In-app How it works — same Setup / Run story for every persona

3. Phase A — Onboard the vendor relationship

Goal: PennyMac can receive and attribute files from a named counterparty without mixing them with another vendor's portfolio.

1. **Identify the counterparty** — correspondent, bulk seller, sub-servicer, internal channel, etc.
2. **Create the Vendor** in Vendors & Mapping (Analyst / SME). Capture identity and status.
3. **Create one or more Feeds** under that vendor. A feed = one file layout (boarding tape vs intraday changes can be separate feeds).
4. **Lifecycle** — SME/Compliance can suspend a vendor. Suspended vendors' files are *recorded* but *not processed*.
5. **Scope the console** — the left-rail vendor selector limits Import Monitor, exceptions, deliveries, and APIs to that vendor.

Integrity: a change file cannot quietly alter another vendor's loans (CHG06 → dual control, servicing transfer only).



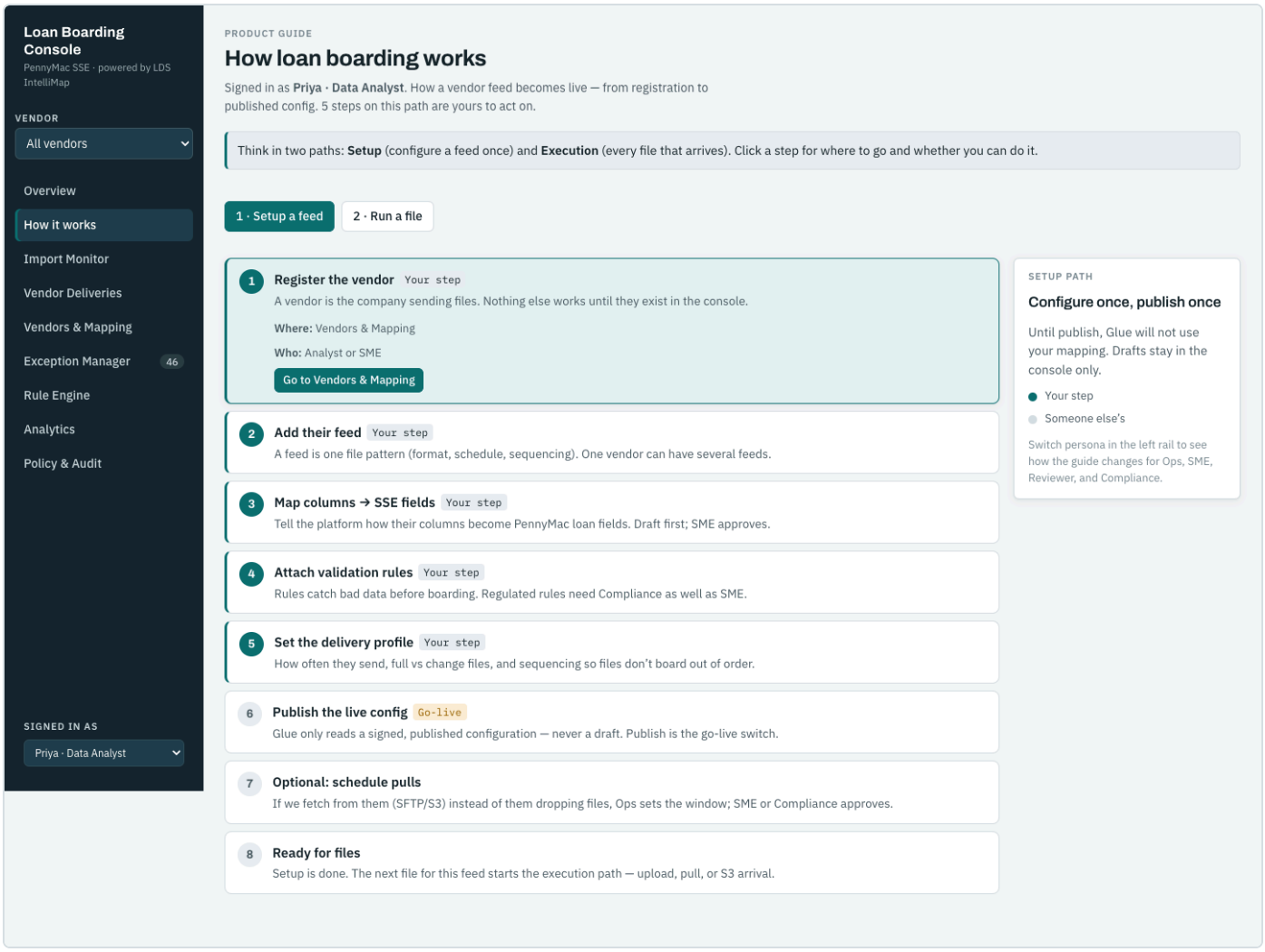
Vendors & Mapping — where onboarding and feed setup live

4. Phase B — Setup the feed (configuration & publish)

Goal: Glue knows exactly how to interpret this feed's files. Until publish, nothing is live.

1. **Mapping** — Analyst maps vendor columns → SSE fields (draft). SME (not the preparer) approves.
2. **Rules** — Attach validation rules. Regulated rules need Compliance as well as SME.
3. **Delivery profile** — full / incremental / snapshot; action codes; clear tokens; protected fields; sequencing. Second SME approves changes.
4. **Publish** — SME publishes the signed configuration ("Glue config vN"). This is the go-live switch.
5. **Optional schedule** — If PennyMac pulls files, Ops proposes a schedule; SME/Compliance approve. Hosts/keys allow-listed.

> **Publish is the contract with Glue.** Drafts and proposed profiles stay in the console only. Glue reads `current.json` (HMAC-signed). A hand-edited or stale bundle is refused and the batch fails as configuration rejected.



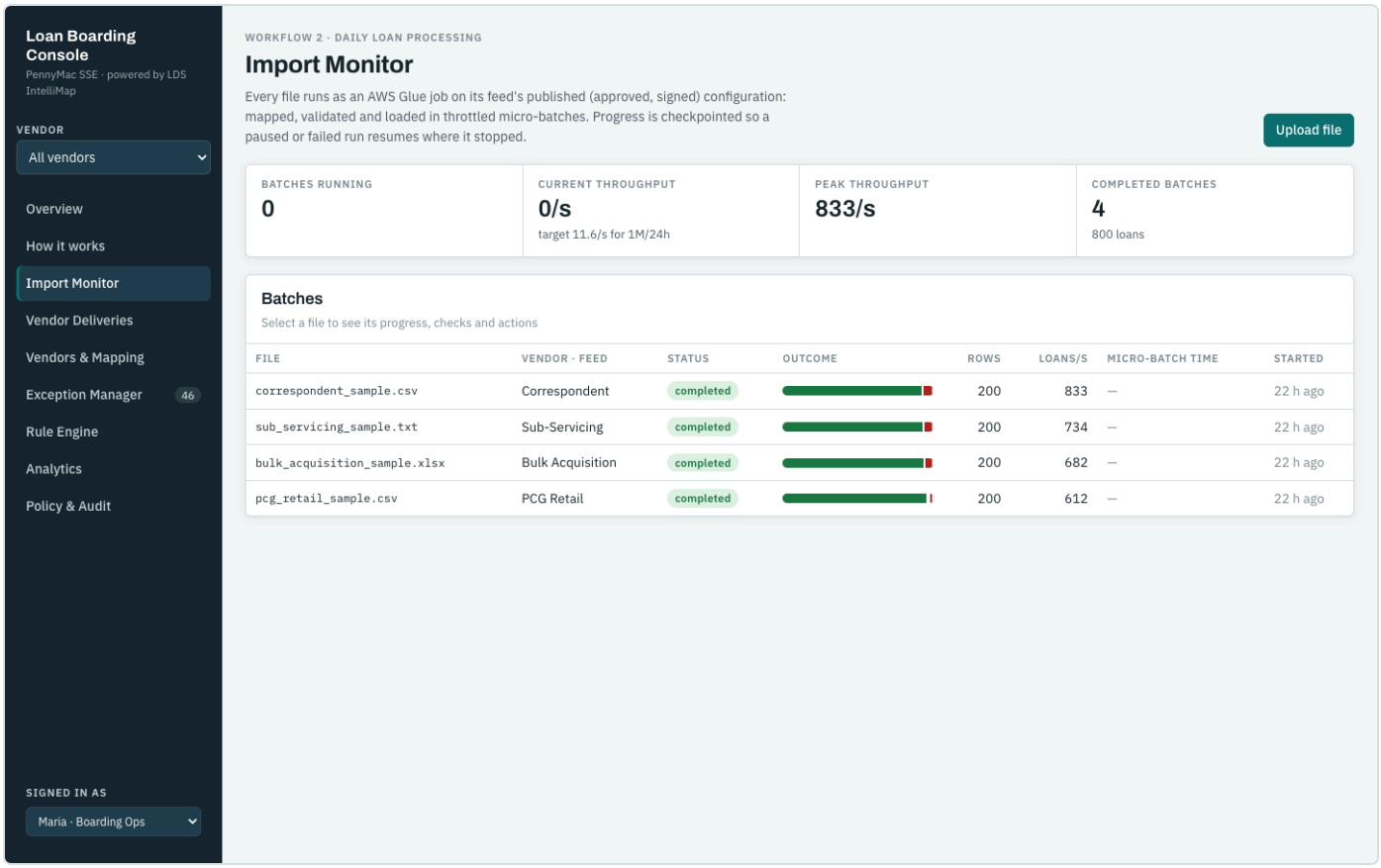
Setup path as Priya · Analyst — Your step chips show what she can do

5. Phase C — How a file is picked up

Goal: Every file, however it arrives, becomes the same kind of **batch** on the same boarding path.

Door	Who / what	What happens
S3 push	Vendor (or gateway) writes to `raw/incoming/<feed>/…`	EventBridge Object Created → Lambda `on_file_arrival` → `create_batch` → dispatch
Scheduled / pull-now	Scheduler tick or Ops/SME Pull now	Glue `lbp-vendor-pull` fetches via SFTP/S3 into raw → same arrival event
Ops upload	Maria · Boarding Ops in Import Monitor	API stores file under uploads/raw → dispatch (same pipeline)

Duplicates are ignored (linked to the prior batch). Bad prefixes are logged and skipped. Load type can come from metadata or the delivery profile (Ops can override on upload).



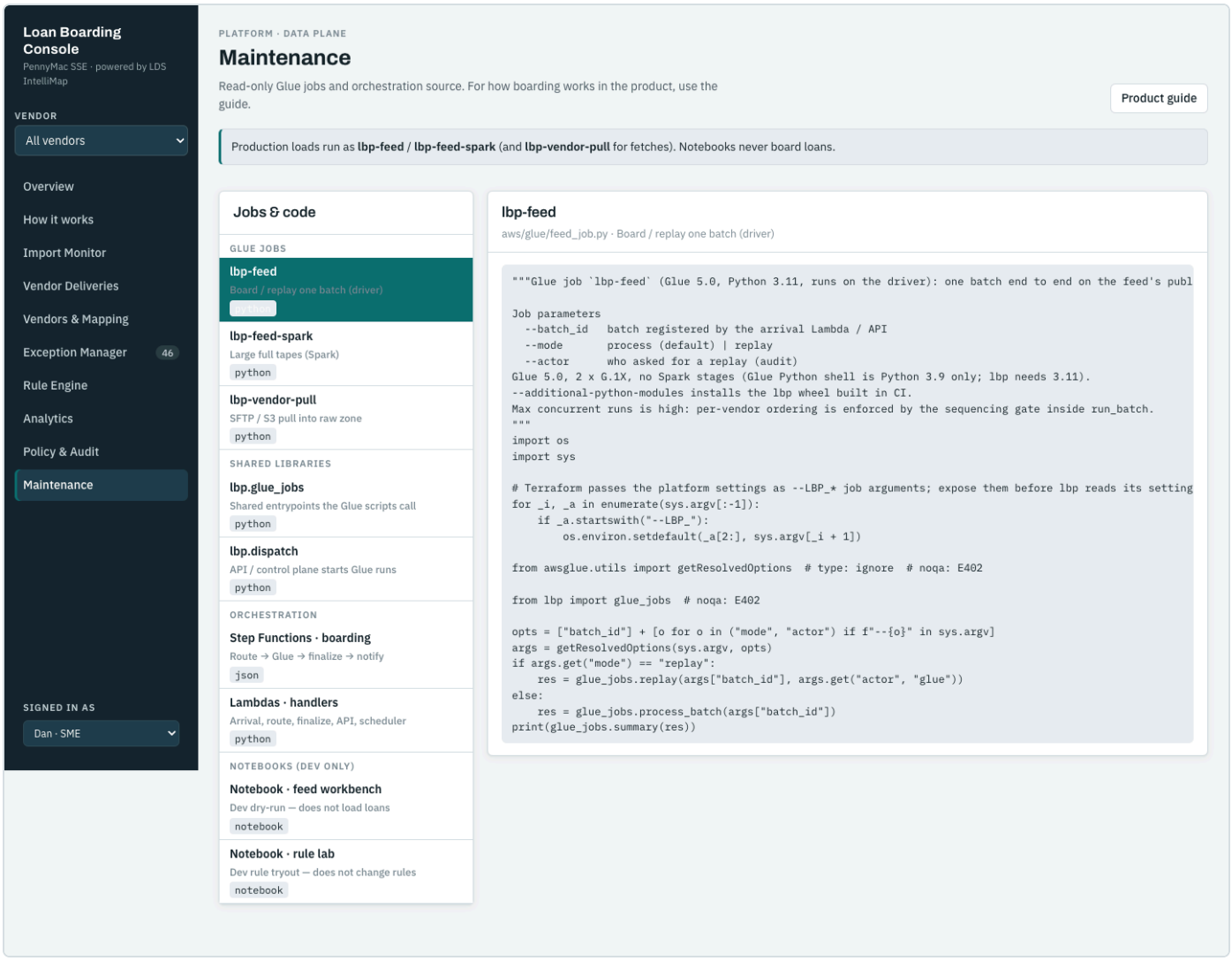
Import Monitor — Ops uploads and watches every batch

6. Phase D — Boarding execution (the run)

Goal: Turn one batch into boarded SSE loans (and held exceptions), safely and restartably.

1. **Dispatch** — AWS: start Step Functions boarding. Local/demo: in-process thread. Same business steps.
2. **Route** — Choose engine: standard `lbp-feed` (default) or `lbp-feed-spark` for very large full text tapes.
3. **Sequencing gate** — Is this the next file for the vendor? If not → waiting / out_of_order (see §14).
4. **Pre-flight** — Schema/drift vs published config. Block → Import Monitor remediation or SME/Compliance override.
5. **Map → validate → triage (T1–T5)** — Good rows proceed; failures become exceptions by tier.
6. **Apply** — Full tape: board new loans. Incremental/snapshot: merge/plan changes (CHG rules as needed).
7. **Checkpoint** — Micro-batches commit; crash/resume continues; Spark failure can fall back to standard.
8. **Reconcile** — Source rows accounted for; advance vendor sequence cursor on success.

```
dispatch → route_batch → Glue (lbp-feed | lbp-feed-spark .sync)
        → finalize_batch → Outcome → SNS
        → release queued followers (dispatch again)
```



Maintenance — Glue jobs and orchestration artifacts (SME/Admin)

7. Phase E — Exceptions, outcomes & notifications

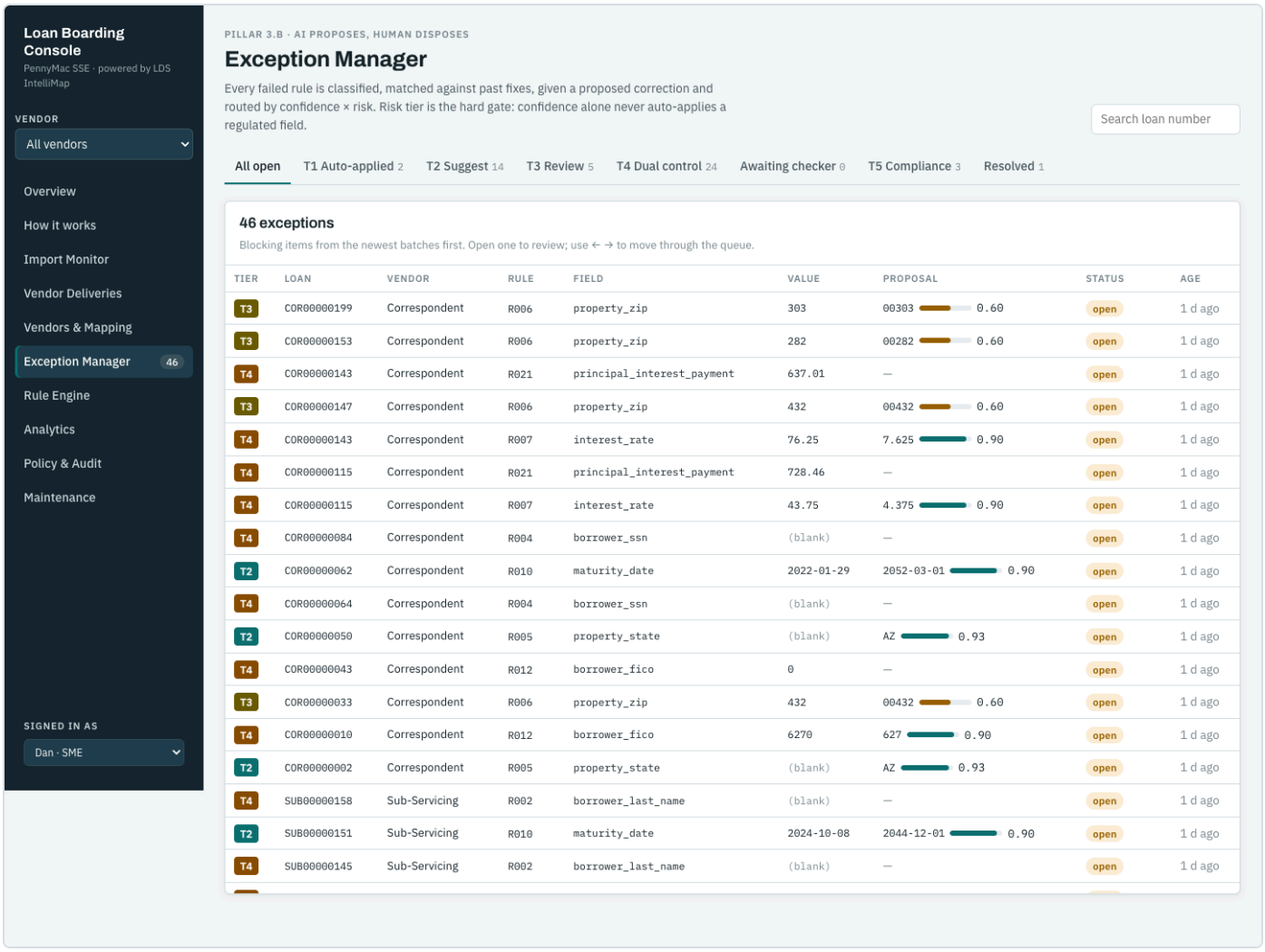
Goal: Humans only touch what needs judgment; everything else is audible and auditable.

Outcomes (Import Monitor / SNS)

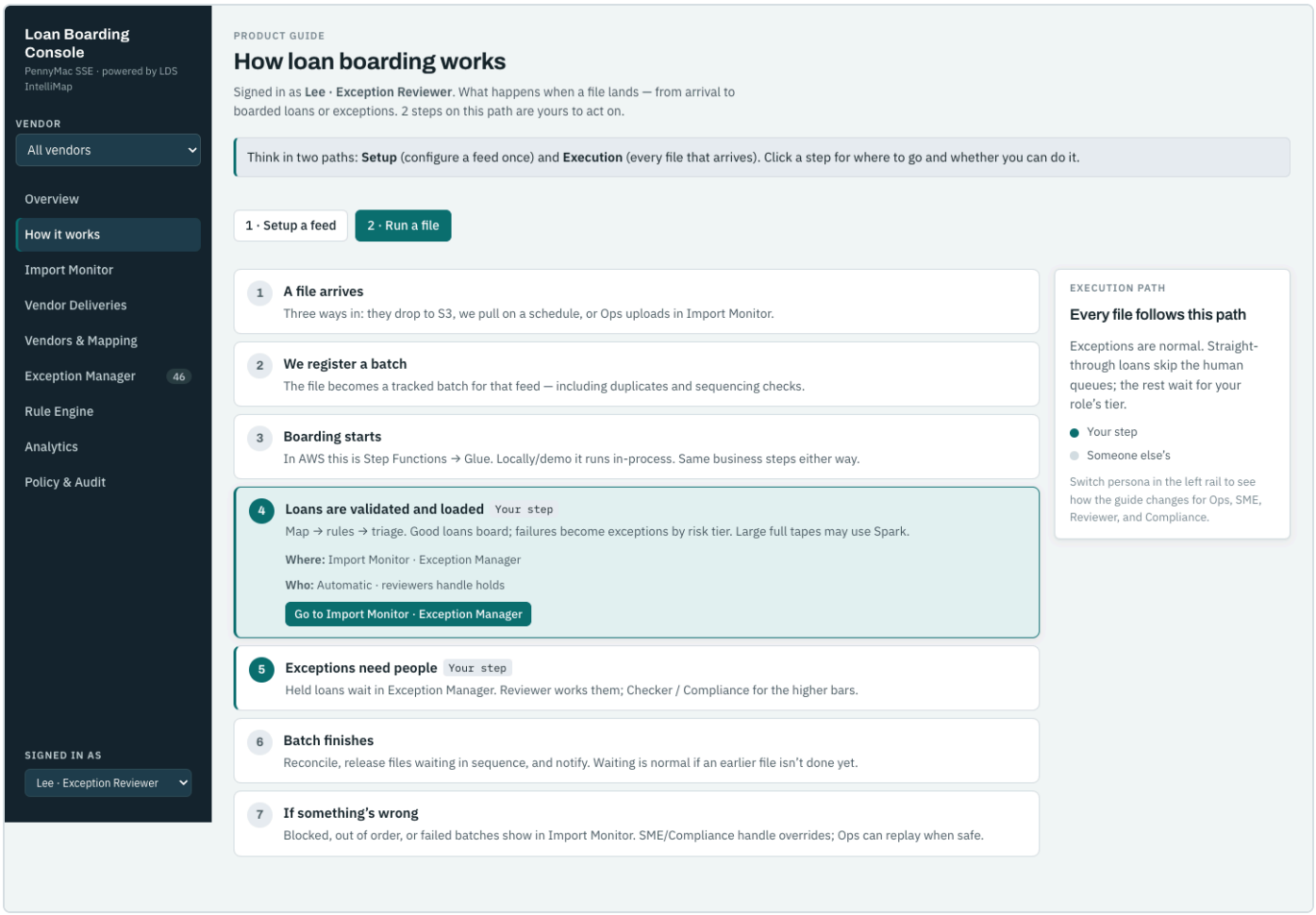
- **completed + balanced** — success notify
- **waiting** — sequencing gate; not a failure
- **blocked / out_of_order / paused** — needs attention
- **failed** — mark_failed + SNS; fix and resume/replay

Exceptions (Exception Manager)

- **T1** — auto (kill switch / rate limit)
- **T2–T3** — Reviewer
- **T4** — Maker + Checker
- **T5** — Compliance only (OFAC/fraud halt)



Exception Manager — human work after Glue triage

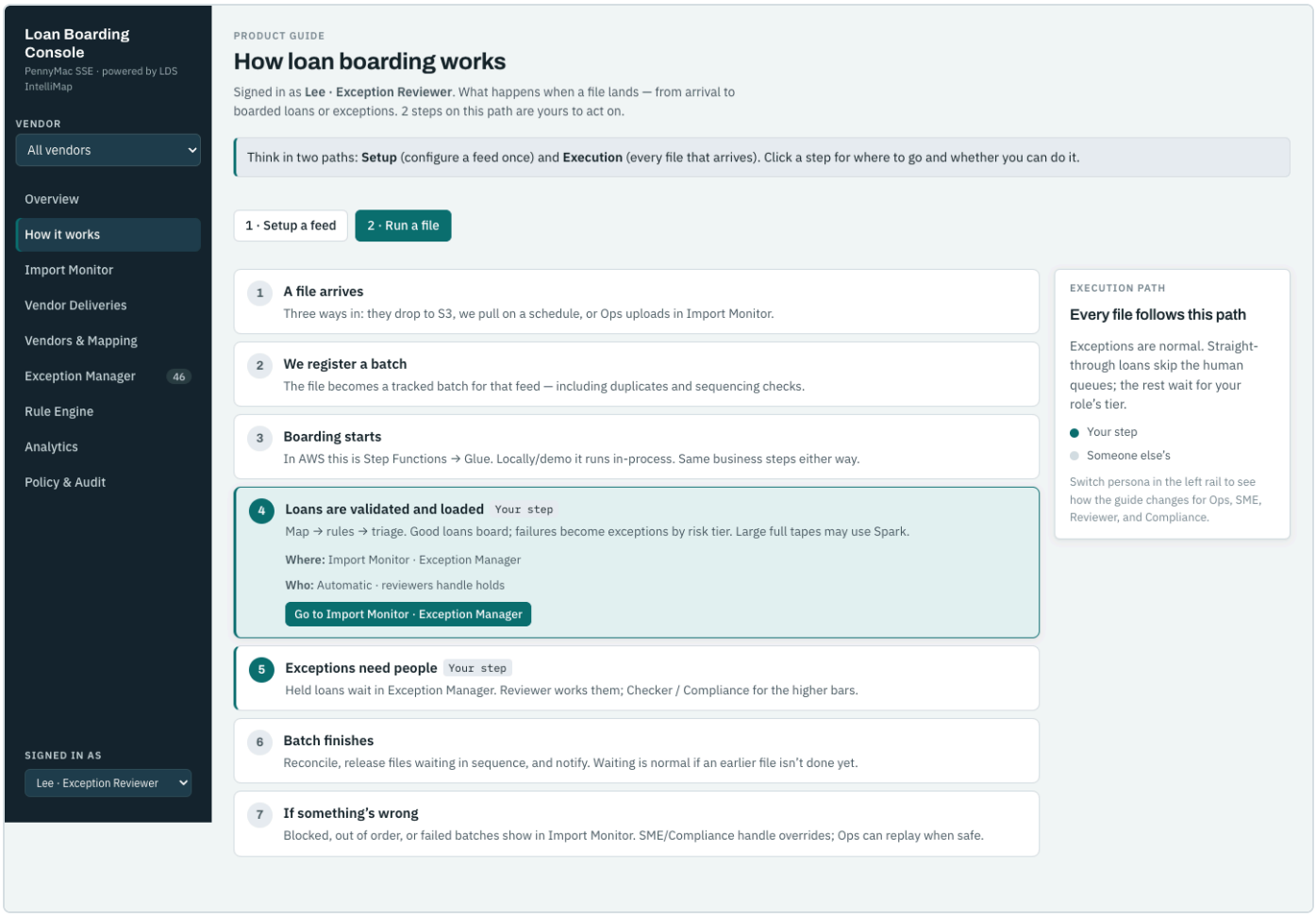


Lee · Reviewer — guide highlights execution steps that are theirs

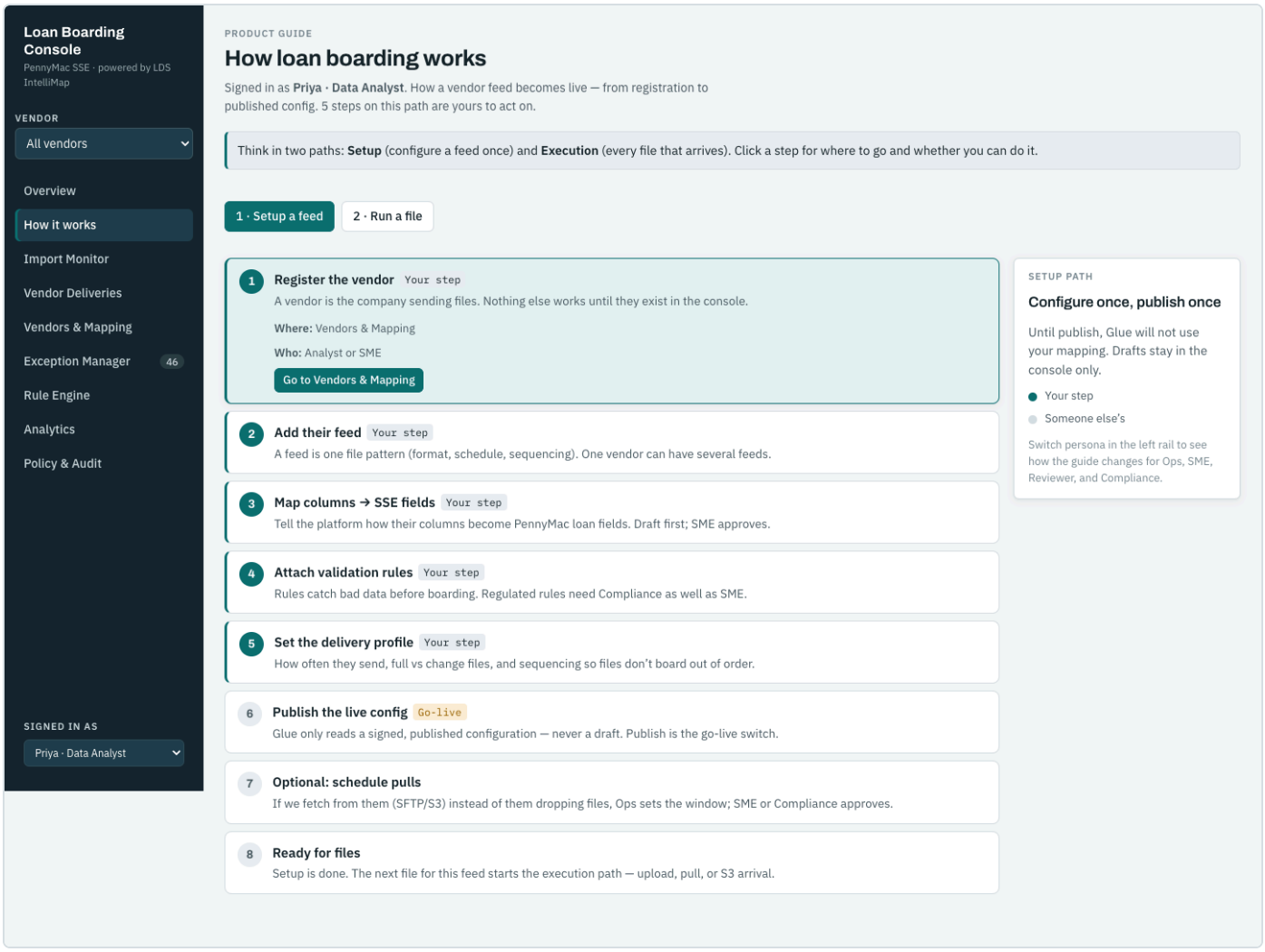
8. Who does what — personas

Demo and local modes switch persona in the left rail. Production uses Cognito groups mapped to the same roles. For functions and how the platform helps each person, see **Appendix A**.

Persona	Roles	Primary job	Default guide path
Priya · Data Analyst	`analyst`	Draft mappings & delivery profiles	Setup a feed
Dan · SME	`sme, analyst`	Approve, publish, rules, vendor lifecycle	Setup a feed
Maria · Boarding Ops	`ops`	Upload, monitor batches, schedules, replay	Run a file
Lee · Exception Reviewer	`reviewer`	Work exception queues	Run a file
Sam · Checker	`checker, reviewer`	Checker approval on dual-control items	Run a file
Ana · Compliance	`compliance, checker`	Regulated rules, policy, T5, audit	Setup a feed
Platform Admin	`admin (+ all)`	Full access + demo setup + Maintenance	Setup a feed



How it works — Lee · Reviewer defaults to Execution (“Run a file”) with Your step chips



How it works — Priya · Analyst on Setup a feed

9. Full capability matrix

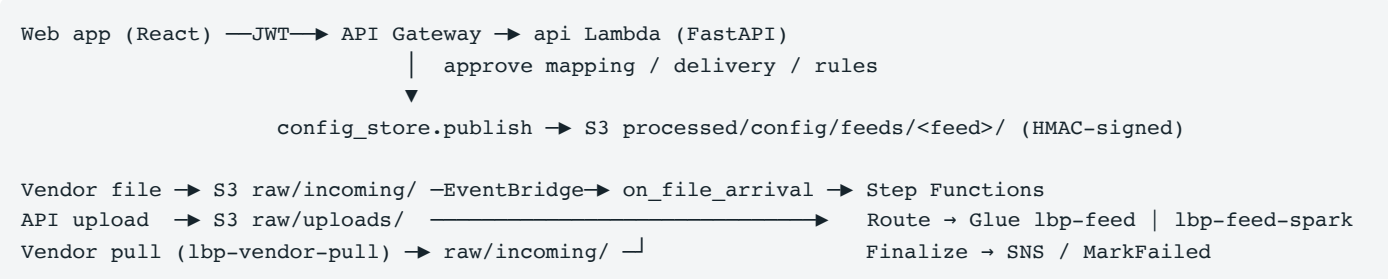
Capabilities mirror backend `ROUTE_ROLES` / `CAPS`. Admin bypasses any role check.

Capability	Roles	Where in console
editMapping	analyst, sme	Vendors & Mapping
approveMapping	sme	Vendors & Mapping
upload	ops	Import Monitor
pauseBatch	ops	Import Monitor
resumeBatch	ops, sme	Import Monitor
overridePreflight	sme, compliance	Import Monitor

replay	ops, sme	Import Monitor
workExceptions	reviewer, checker, compliance	Exception Manager
checkerApprove	checker, compliance	Exception Manager
complianceAct	compliance	Exception Manager (T5)
editRules / authorRules	sme, compliance (+ analyst author)	Rule Engine
editPolicy	compliance	Policy & Audit
demoSetup	admin	Policy & Audit / Overview
editVendors / vendorLifecycle	analyst/sme · sme/compliance	Vendors & Mapping
moveFeed / publishConfig	sme	Vendors & Mapping
editDelivery / approveDelivery	analyst/sme · sme	Vendors & Mapping
skipGap / applyOutOfOrder / discardBatch	ops/sme · sme/compliance · ops/sme	Import Monitor
snapshotReviews	sme	Import Monitor / Exceptions
schedules / approveSchedule	ops/sme · sme/compliance	Vendor Deliveries
readAudit	compliance, sme	Policy & Audit
readMaintenance	sme, admin	Maintenance

10. Architecture (control plane vs data plane)

End-to-end path



Why this split

Concern	Decision
All data integration on Glue	Every byte of loan data is parsed, mapped, validated and loaded inside Glue. Lambdas only register, route, report and serve the API.

Driven by the web app	Glue jobs take `--batch_id` (or `--schedule_id`). Everything else comes from the published feed configuration.
Drafts never run	Draft mapping / proposed delivery profile is not published. Glue reads `current.json`.
Tamper-evident	Each bundle has SHA-256 + HMAC. Tampered/stale config → batch fails with audited `batch.config_rejected`.
Live safety switches	T1 kill switch, auto-suspension, vendor suspended status are read from DB at run time.
Same code everywhere	The `lbp` wheel runs in Glue, local worker and tests.

11. Signed configuration & publish events

Belongs with Phase B (setup). Glue never boards from a draft.

Event	Published
Mapping version approved (SME ≠ preparer)	that feed
Delivery profile approved (second SME)	that feed
Rule enters/leaves active/canary/shadow or retires	every live feed
Feed moved to another vendor	every live feed
migrate Lambda on deploy	any live feed missing a bundle
Republish (SME + reason)	that feed

UI shows “Glue config vN” chips with version, mapping version, rule counts, signature status, sync with active mapping, and history.

12. Glue jobs & orchestration

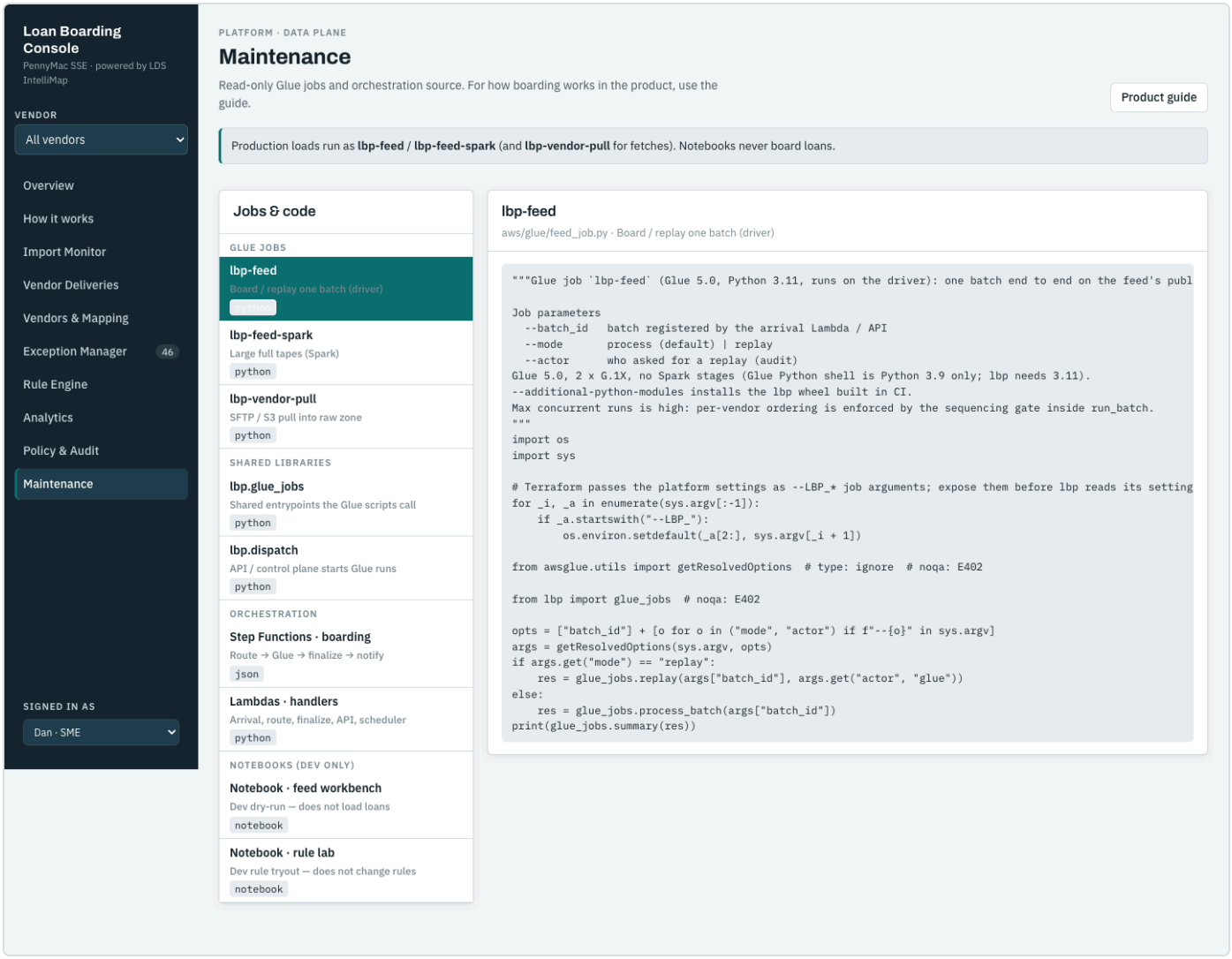
Belongs with Phase D (boarding run).

Job	Runtime	Used for
`lbp-feed`	Glue 5.0 Python 3.11, driver	Default every batch; also replay
`lbp-feed-spark`	Glue Spark	Very large full CSV/TXT/PSV/DAT; fallback to standard on failure
`lbp-vendor-pull`	Glue 5.0 driver	Fetch into raw zone only

Step Functions boarding (ASL)

```
Route (route_batch) → Engine choice
  → GlueStandard (lbp-feed .sync) | GlueSpark (lbp-feed-spark .sync)
  → Spark TaskFailed may fall back to GlueStandard
  → Finalize (finalize_batch) → Outcome
       completed+balanced → NotifyCompleted (SNS)
       waiting → Succeed (sequencing gate)
       out_of_order / blocked / paused → NotifyAttention
       else → NotifyFailed → Failed
  Catch → mark_failed → SNS → Failed
```

Notebooks under `aws/glue/notebooks/` are development only — they never board loans.

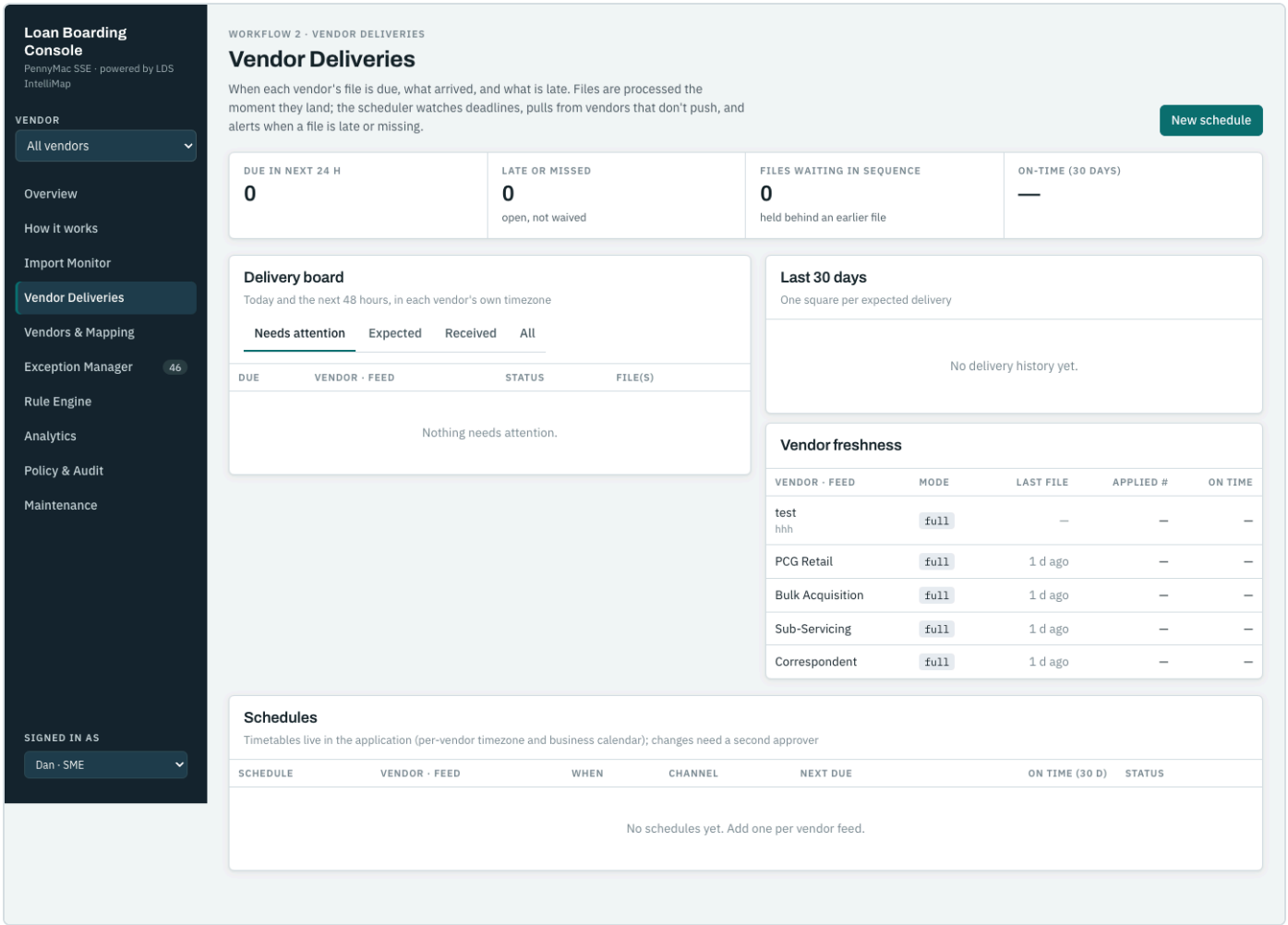


Maintenance — Glue / SFN / Lambda source (SME)

13. Delivery scheduler & vendor pull

Belongs with Phase C (file pickup). Arrival stays **event-driven**; the scheduler watches deadlines and pulls — it never delays a file that already landed.

- EventBridge Scheduler ~every 5 minutes → `scheduler_tick` (reserved concurrency 1).
- Cron in vendor IANA timezone (DST-safe), Fed holidays (skip / next business day).
- Options: grace, miss-after, early accept, files per delivery, filename filter, load type for pulls.
- Delivery states: expected → received | late → missed → received_late; waived stops alerts.
- Due pulls start Glue `lbp-vendor-pull` (SFTP pinned host key or S3). Files land in `raw/incoming/` and follow normal arrival.
- Pull-now from console for Ops/SME with schedule rights. Manual: `python -m lbp.cli tick`.
- Leader lease; alerts stamped once; dead-man's-switch if no tick for 15 minutes.
- SFTP: host key mandatory; credentials from Secrets Manager; allow-listed hosts/CIDRs; never delete remote.



Vendor Deliveries — schedules and pull windows

14. Vendors, feeds, incremental loads & sequencing

Vendors and feeds

- A **vendor** is the counterparty; a **feed** is one file layout with its own mapping, delivery profile, schedules and numbering.
- Sidebar vendor scope filters operational pages and APIs (`?vendor_id=`).
- Suspended vendor files are recorded but not processed.
- CHG06: change file touching another vendor's loan → dual control (servicing transfer only).

Load types

Type	Meaning	Platform behaviour
full	Complete boarding tape	New loans; DUP02 re-board protection
incremental	Add / update / close	Merge onto live SSE, then full rules
snapshot	Whole current portfolio	Adds/updates/unchanged + missing → CHG04

Change application (summary)

- Blank = no change (or clear if profile says so); `clear_token` empties a field.
- Identical to SSE → unchanged (slim marker only).
- Protected fields → CHG01 (T4 dual control). Set-once originals never overwritten by change files.
- Optimistic concurrency via `lbp_version`; conflicts → CHG05.
- Closes set `lbp_status=closed`; never delete. All applied changes in `loan_changes`.

File order & sequencing outcomes

Sequence from file name regex or column. Gate + lease in one transaction — exactly one "next" file under concurrency. Vendor cursor (`sources.last_sequence`) only moves forward on completed batches.

Situation	Status	Resolution
Earlier file of this vendor still pending	`waiting`	Released when that file completes
Integer gap (44 arrives, 43 missing), strict profile	`waiting`	43 arrives; or Ops skips gap with reason
File not newer than last applied	`out_of_order`	SME discards or applies with every change dual-controlled
Same sequence as a pending file	Refused at arrival	Discard the pending one first

15. Exception tiers and change-file rules

Belongs with Phase E (finish / human work).

Tiers (confidence × risk)

Tier	Who acts	Notes
T1	Auto-apply (kill switch / rate limit)	AI never on regulated fields
T2–T3	Reviewer	Standard exception work
T4	Maker + Checker	Protected / dual control
T5	Compliance only	OFAC / fraud — loan halted

Change-file exception codes

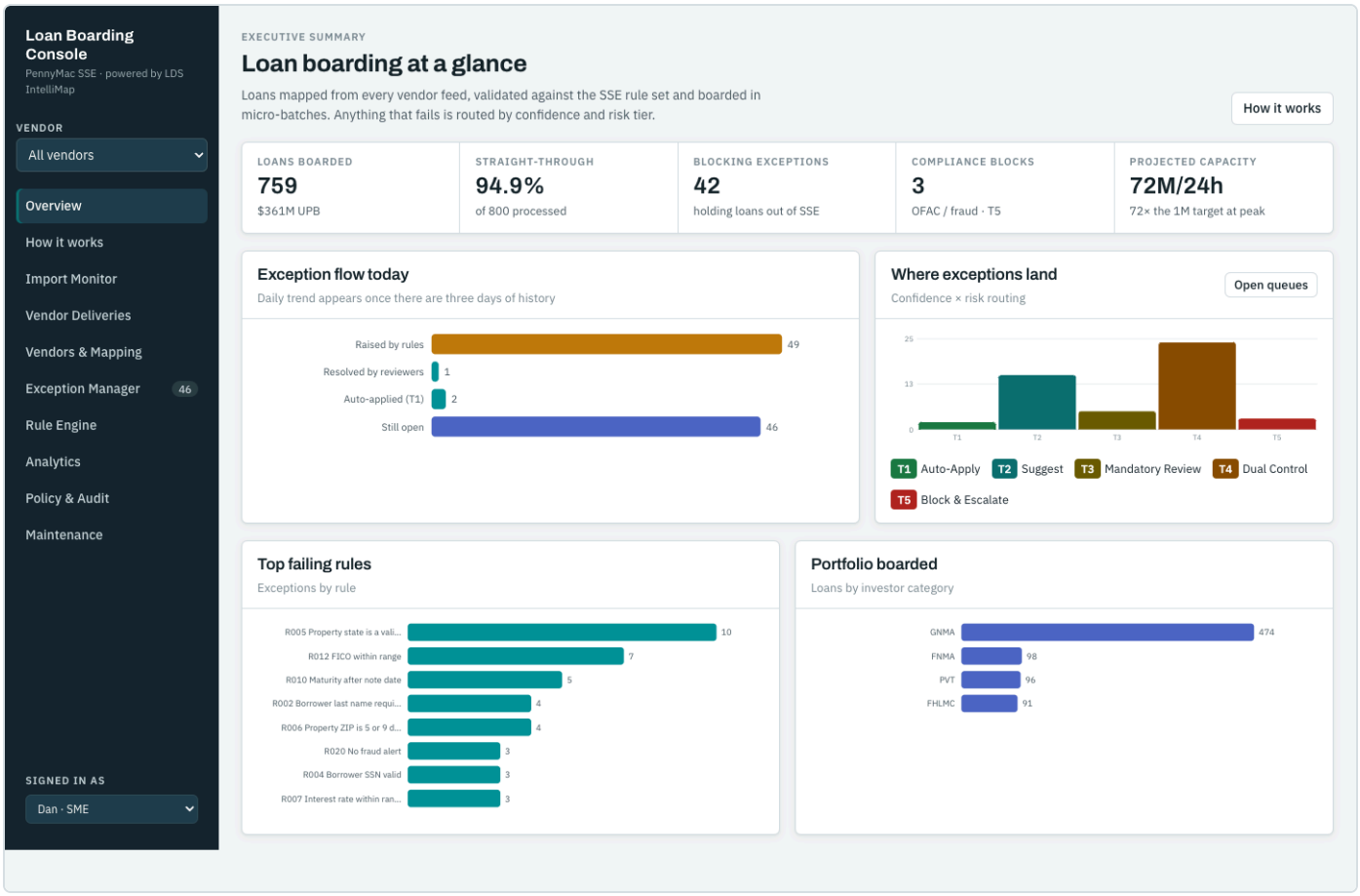
Rule	When	Resolution
CHG01	Protected field changed	Apply vendor / keep SSE / reject (T4)
CHG02	Update/close for loan not in SSE	Correct loan number
CHG03	Change for closed loan	Reopen+apply or reject
CHG04	Loan missing from snapshot	Confirm closed or keep active
CHG05	Concurrent change	Keep SSE or apply vendor
CHG06	Other vendor's loan	Servicing transfer dual control or reject
ACT01	Unknown action code	Default action or reject; update profile

16. Console tour (every screen)

Screenshots of the live product after the process story. Use §1–7 to learn the flow; use this section to find the screen.

16.1 Overview

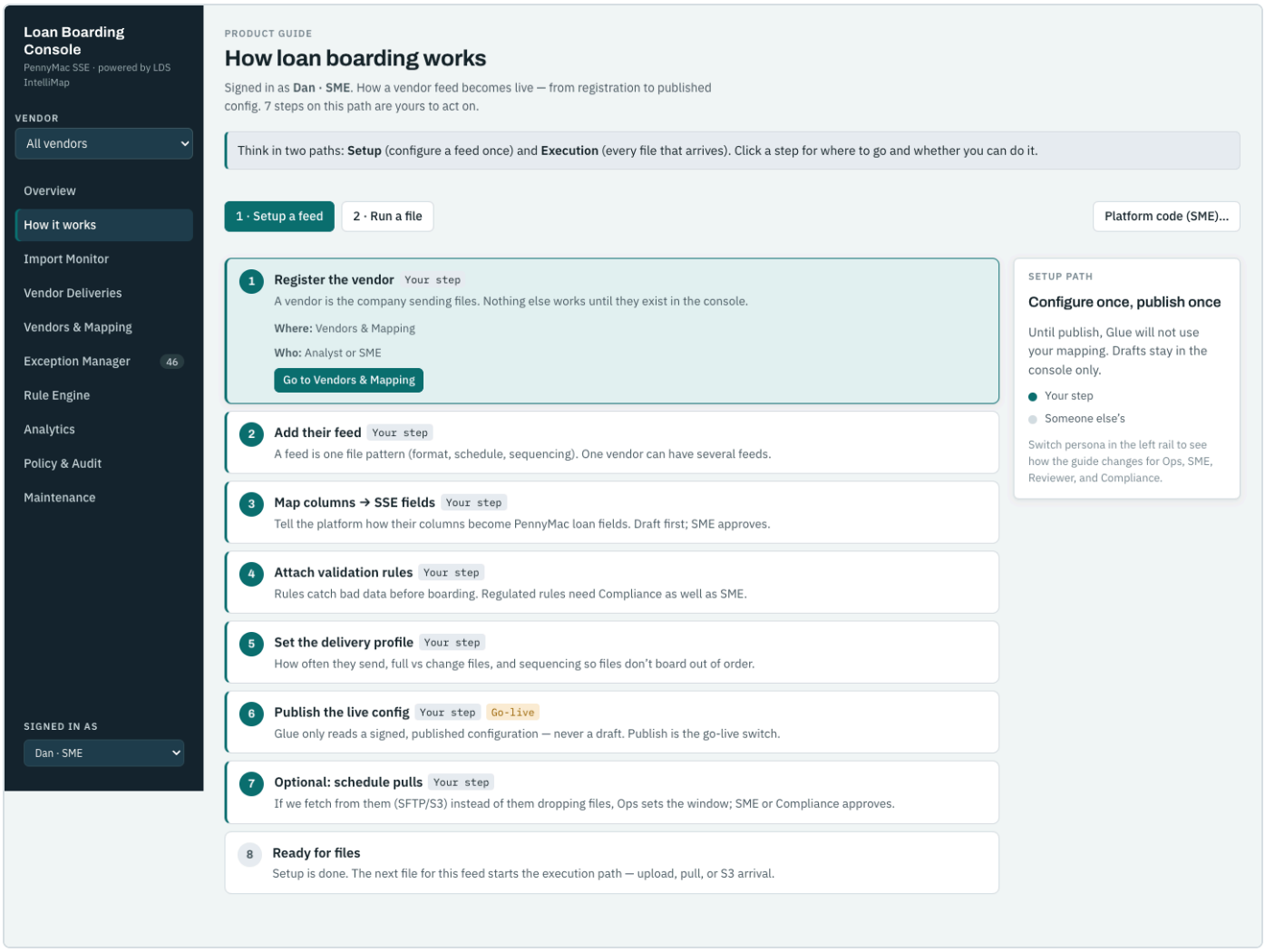
Executive KPIs: loans boarded, straight-through rate, blocking exceptions, compliance blocks, projected capacity. Charts for exception flow and investor mix. Shortcuts by persona.



Overview

16.2 How it works

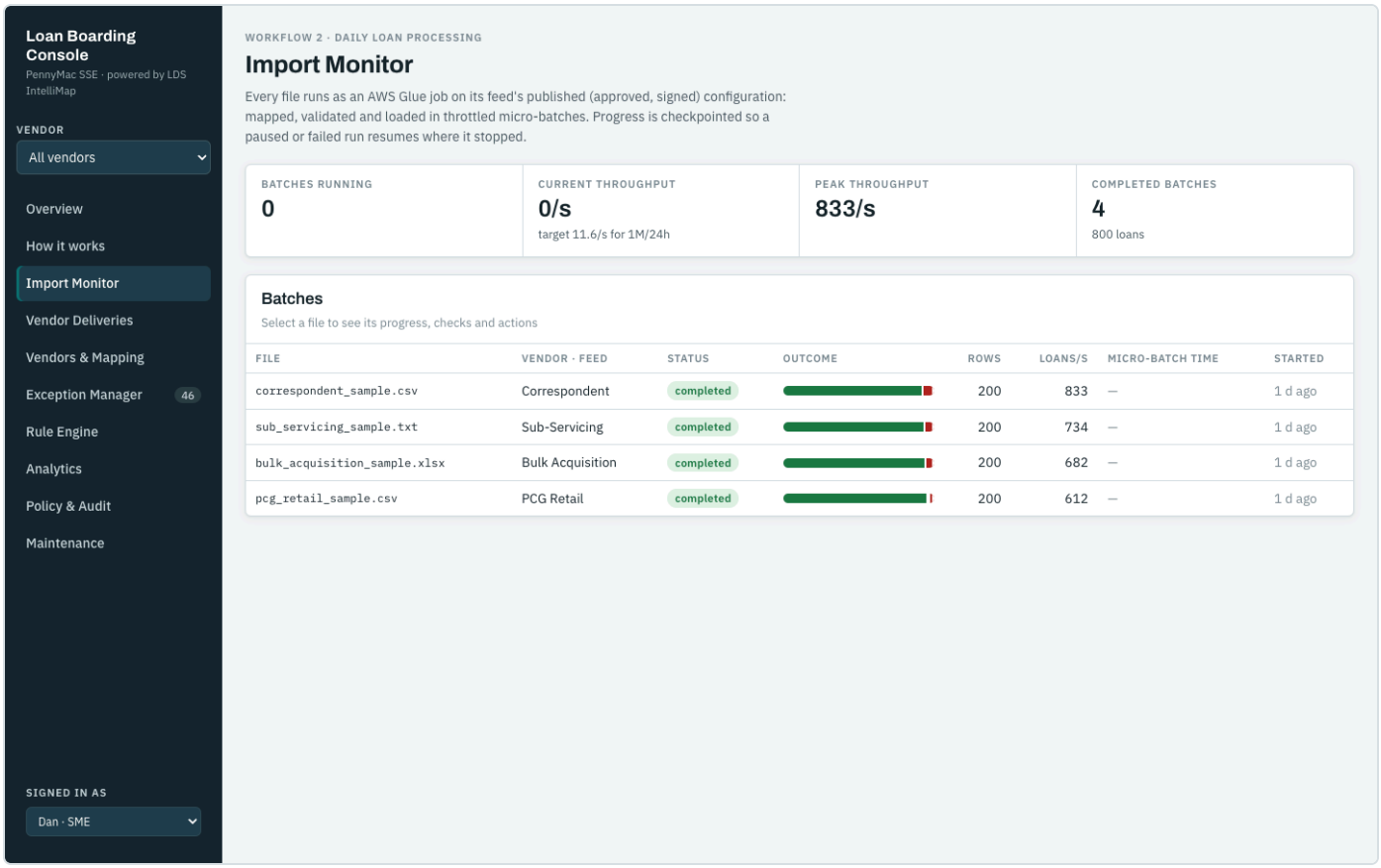
In-app product guide for Phases A–E. Two tabs: Setup a feed / Run a file. Persona-aware “Your step” and Go-to CTAs. SME/Admin link to Maintenance.



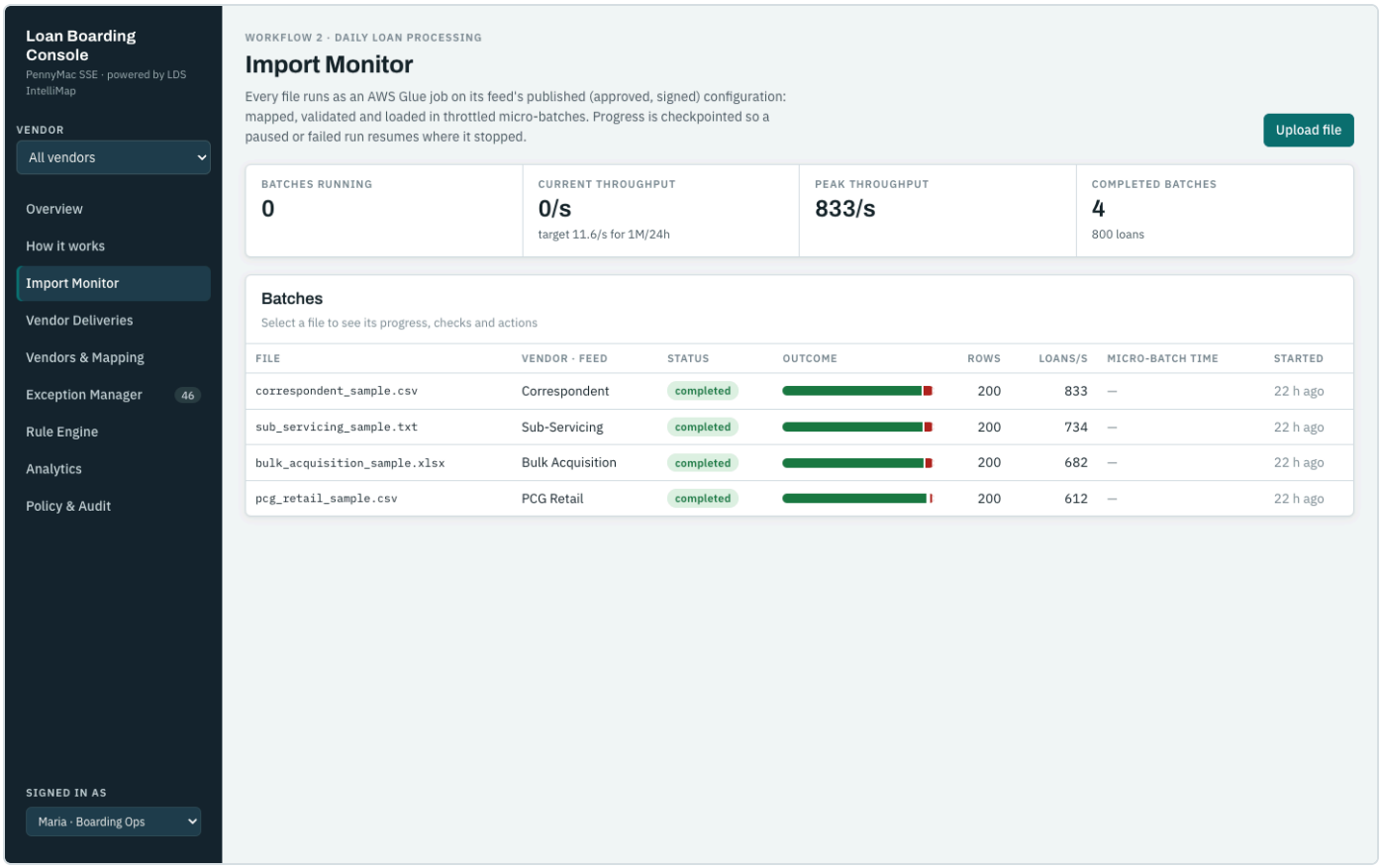
How it works

16.3 Import Monitor

Batches: upload, pause/resume, pre-flight override, sequencing skip/out-of-order, replay, discard, loan change log.



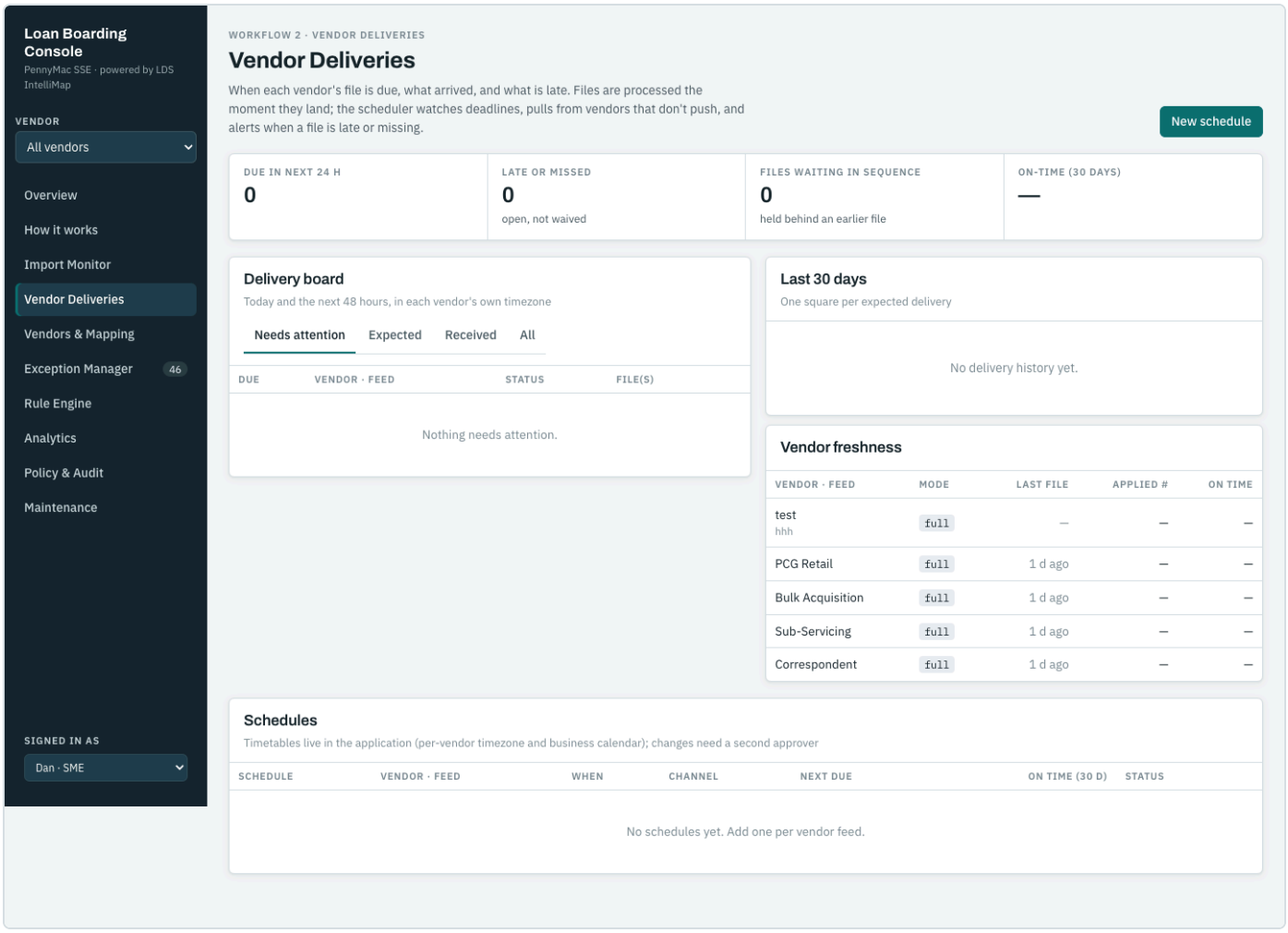
Import Monitor (SME)



Import Monitor — Maria · Ops

16.4 Vendor Deliveries

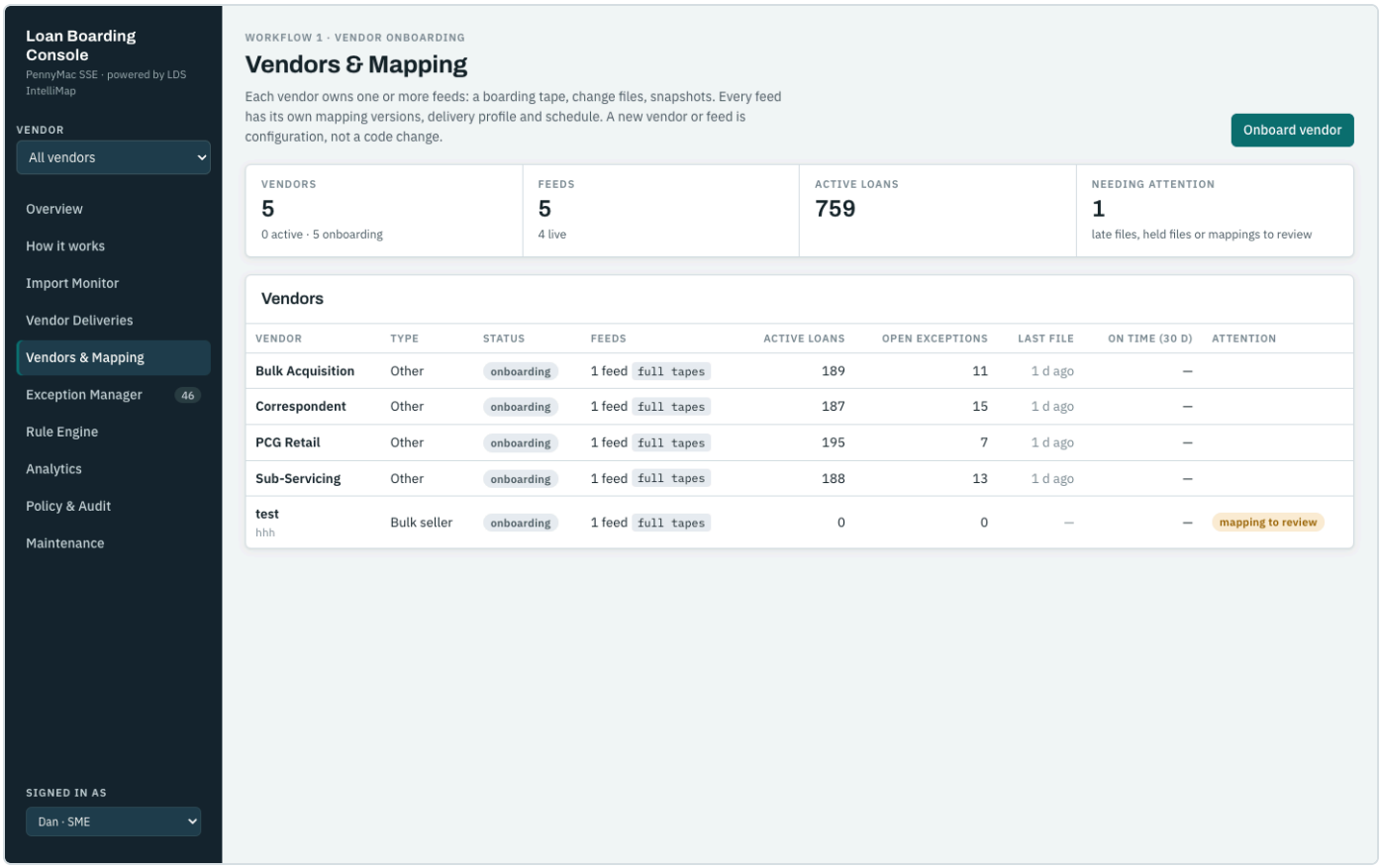
Schedules, windows, late/missed alerts, pull-now, waivers, maker-checker on schedule changes.



Vendor Deliveries

16.5 Vendors & Mapping

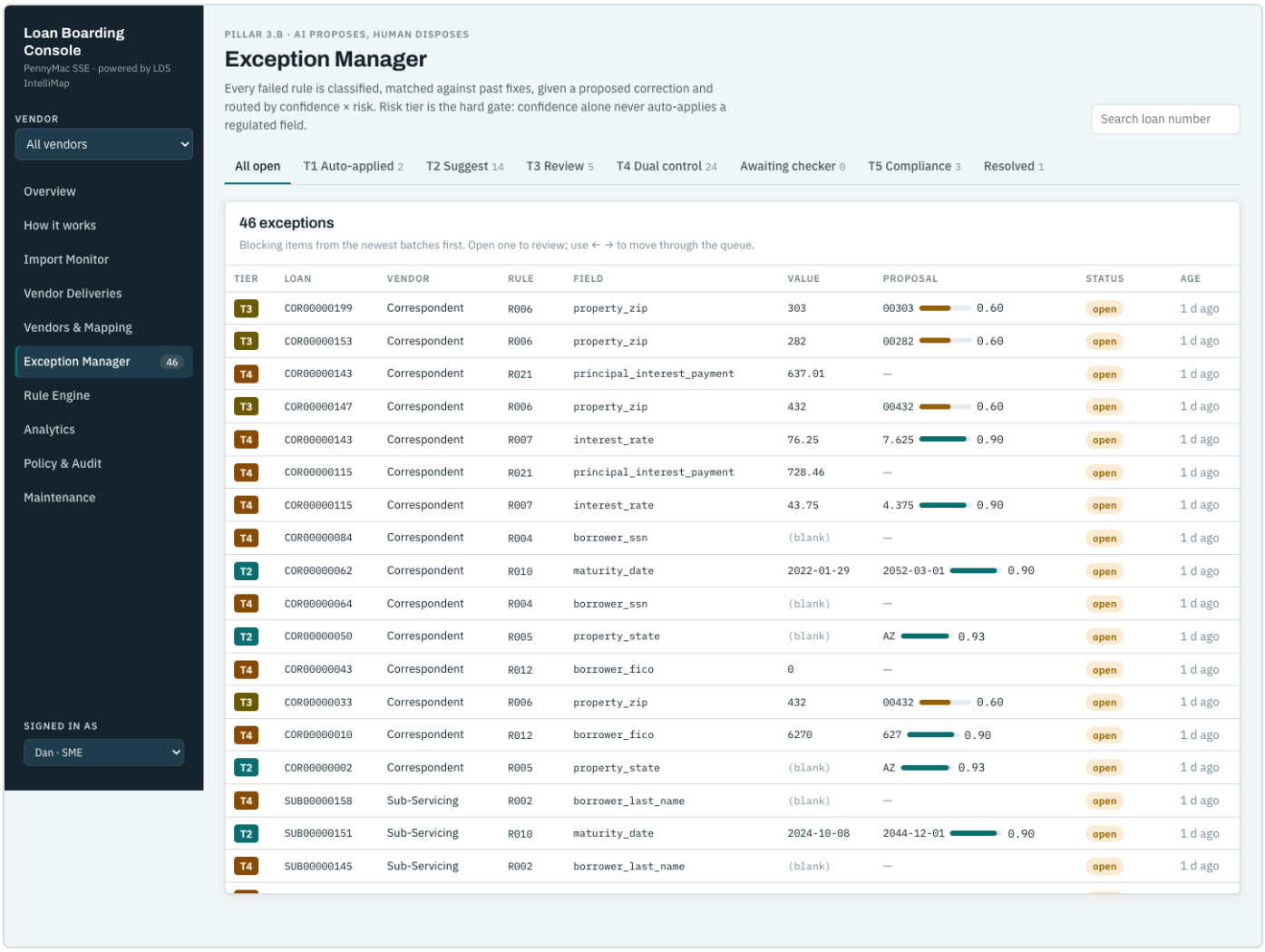
Vendors and feeds, mapping workbench, field inspector, validation preview, version diff, delivery profile, Glue config chip.



Vendors & Mapping

16.6 Exception Manager

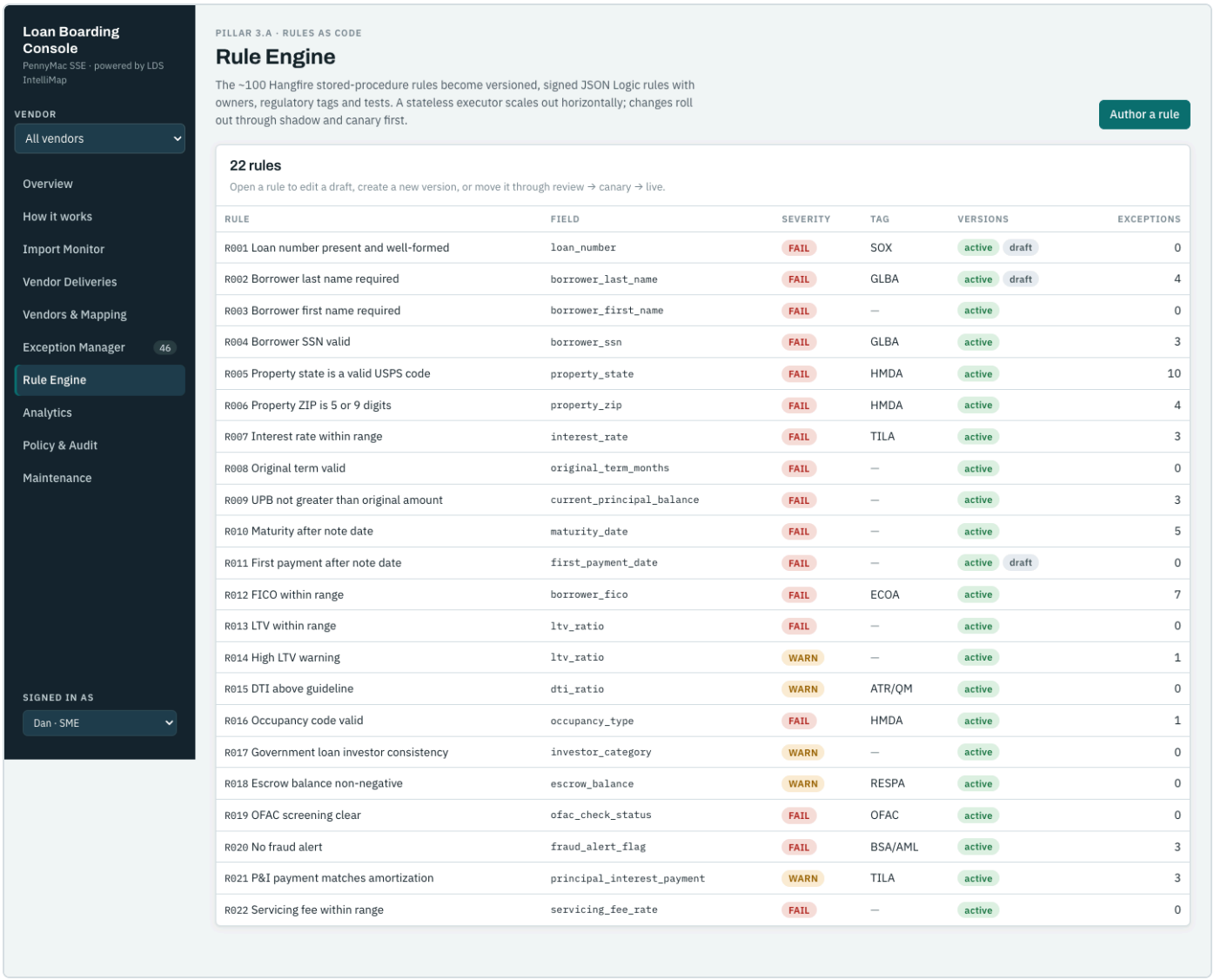
Queues by tier T1–T5. Reviewer / checker / compliance actions with reasons.



Exception Manager

16.7 Rule Engine

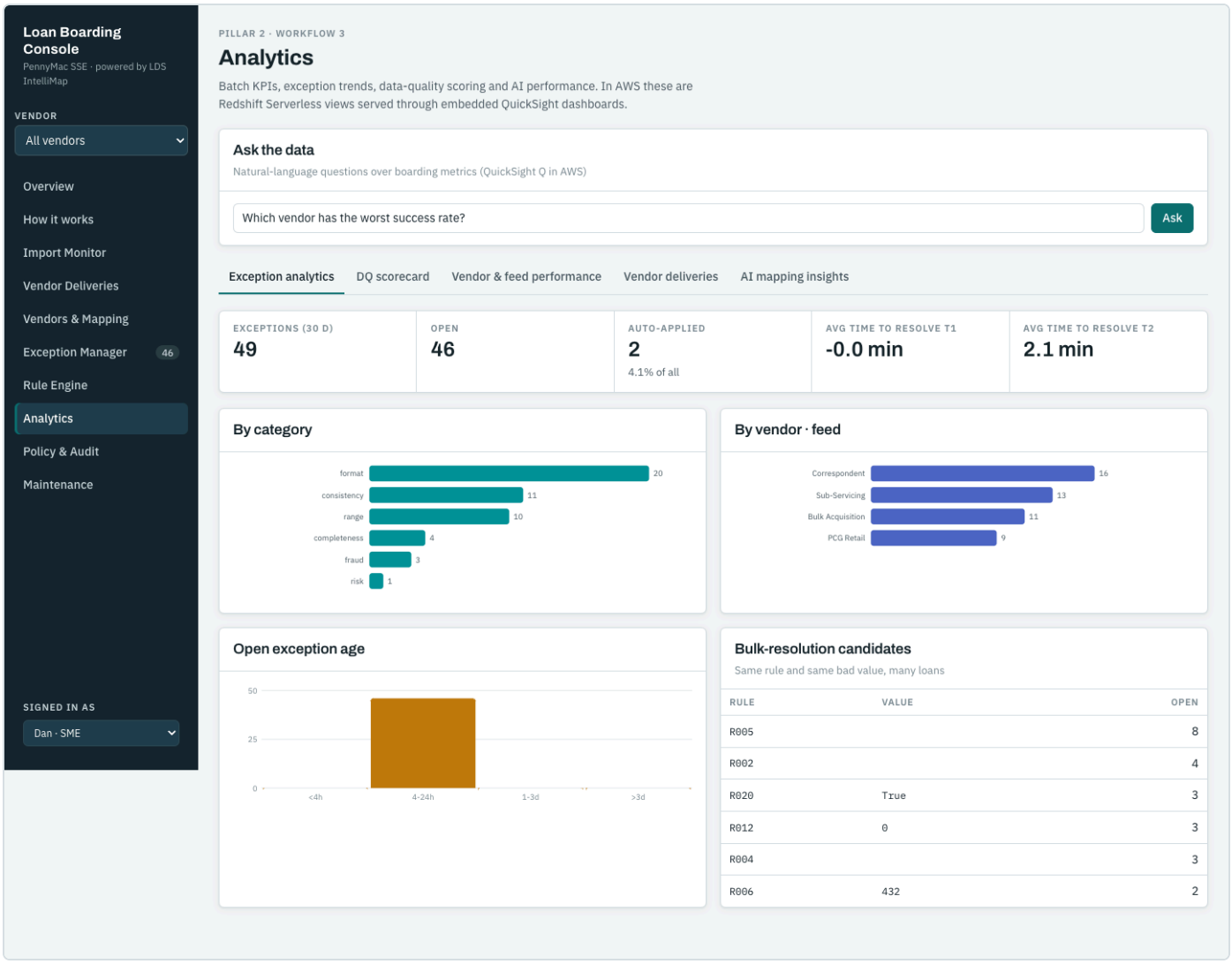
Author in plain English, draft edit, lifecycle transitions. Regulatory tags force Compliance.



Rule Engine

16.8 Analytics

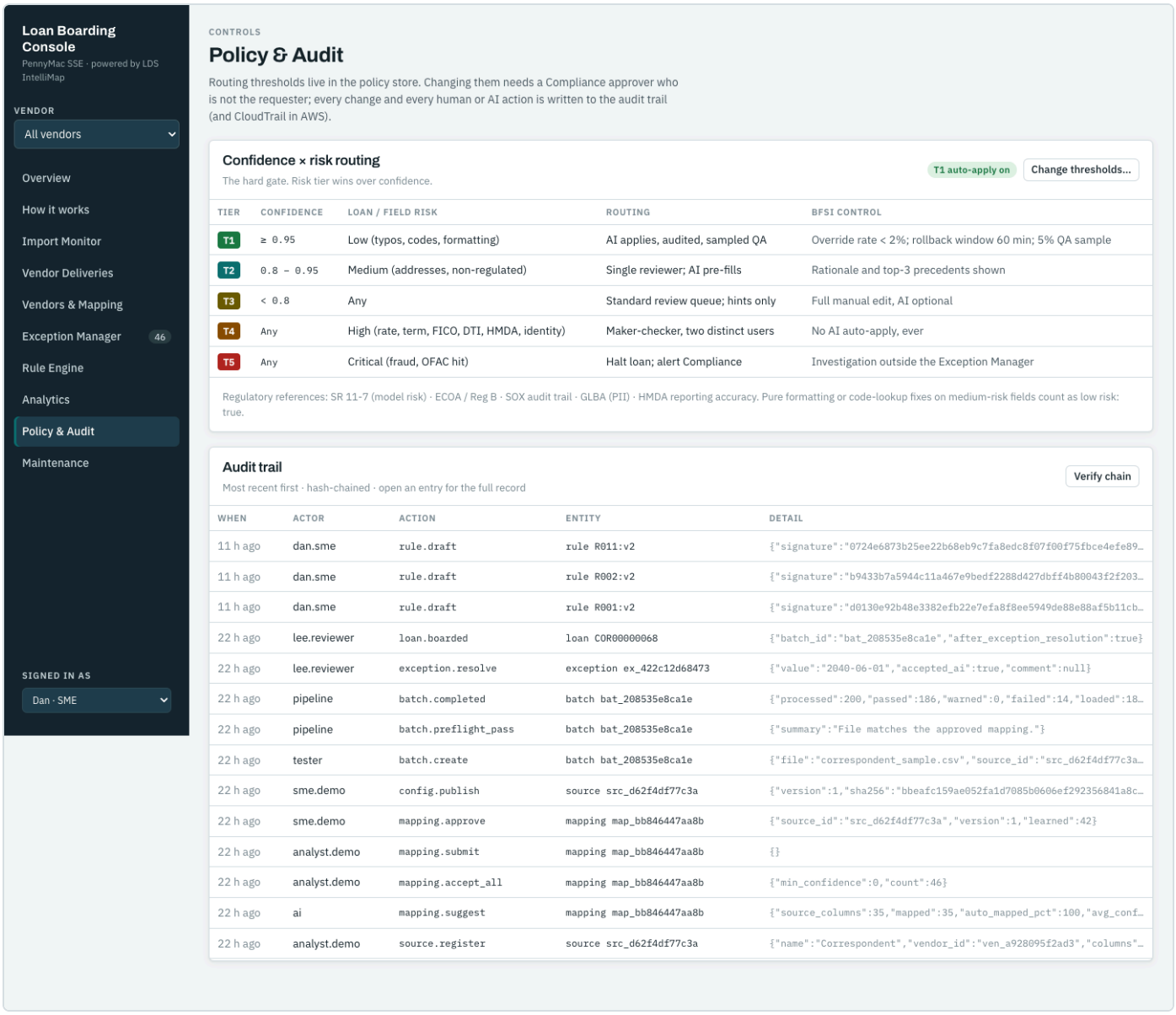
Executive and exception analytics; ask-style questions where enabled.



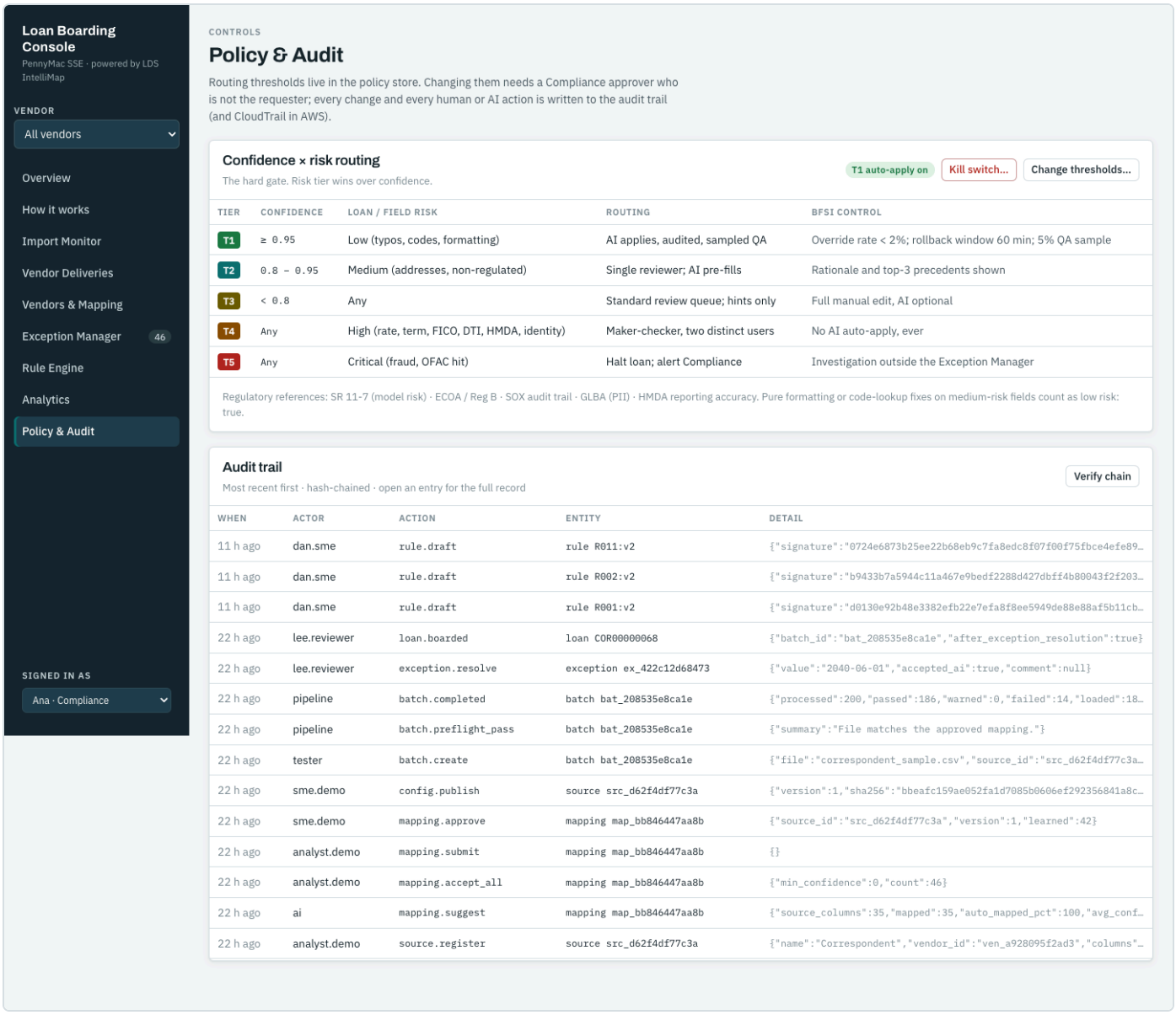
Analytics

16.9 Policy & Audit

Routing policy, T1 kill switch, hash-chained audit, demo scenario (admin).



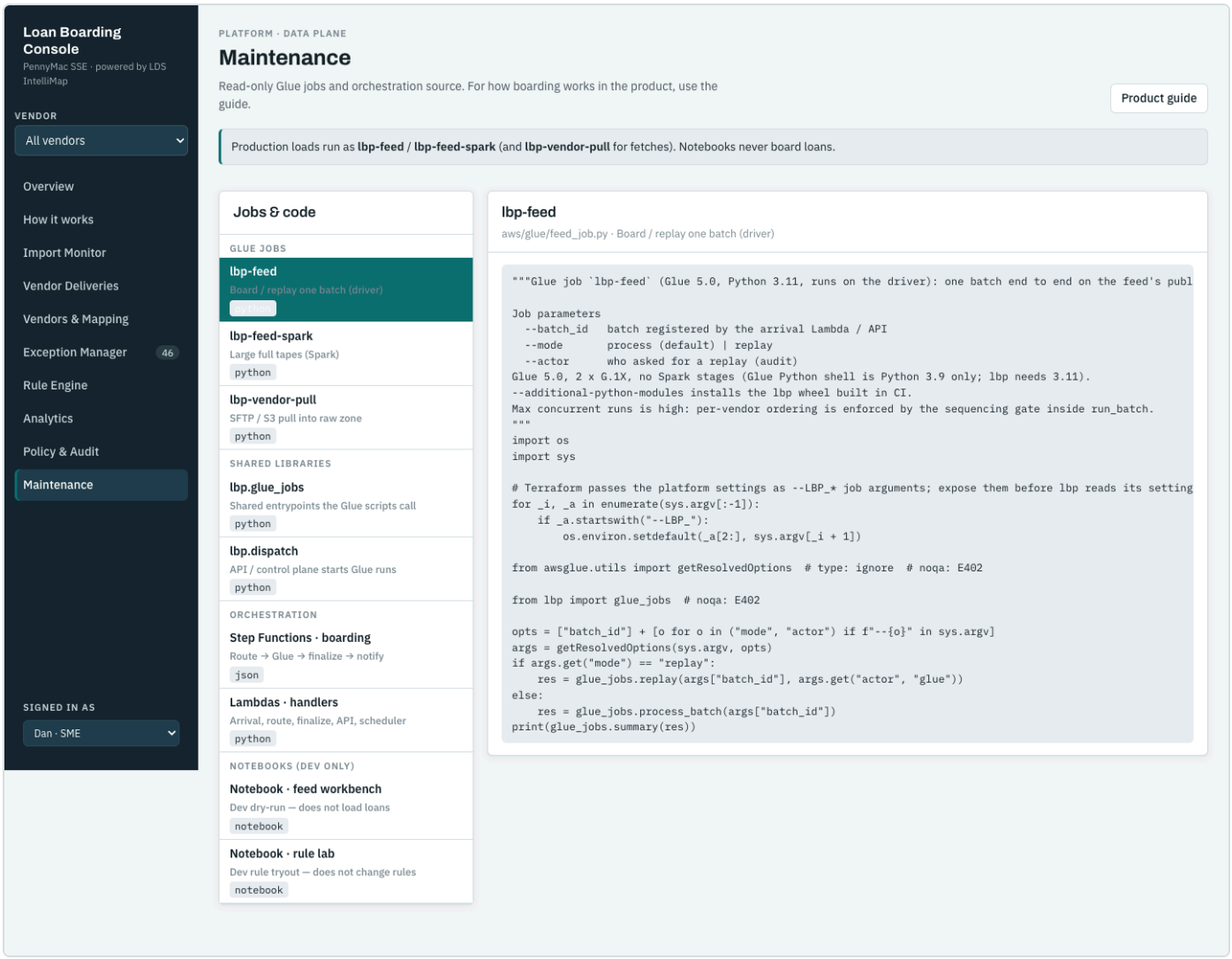
Policy & Audit



Policy & Audit — Ana · Compliance

16.10 Maintenance

Read-only Glue jobs, shared libraries, Step Functions ASL, Lambdas, notebooks. SME/Admin only.



Maintenance

17. Security, compliance & production readiness

Requirement	Control
SOX	Maker-checker; signed configs; hash-chained audit (+ CloudTrail)
GLBA / NPI	KMS, TLS, role masking, retention purge, no NPI in logs/SFN
SR 11-7	No AI auto-apply on T4/T5; T1 kill switch & override-rate suspend
ECOA/HMDA/TILA/RESPA	Regulatory tags; Compliance approval; dual control on sensitive fields
OFAC / BSA-AML	T5 halt; Compliance only
Least privilege	Cognito groups; scoped IAM; limited egress
Credentials	Secrets Manager rotation; OIDC deploys; no long-lived keys in TF

Edge	WAF, rate limit, geo, CSP/headers

Status: ready for UAT in PennyMac AWS; go-live needs account-specific items outside the codebase.

18. Operator runbook

Situation	What to do
Batch **blocked** by pre-flight	Import Monitor → mapping fix + SME approve + resume; or SME/Compliance override with reason
Batch stuck **running**	Auto-requeue after 10 min without heartbeat; `python -m lbp.cli recover`
Reconciliation **mismatch**	SNS alert; compare staged/boarded/held/rejected; LOAD/SCHEMA exceptions
T1 auto-suspended	Policy & Audit; re-enable needs two Compliance users
Emergency stop of AI fixes	Compliance sets `t1_enabled=false`
Audit integrity	`python -m lbp.cli verify-audit` (daily)
Retention	`python -m lbp.cli purge` (daily)
Glue run **failed**	Fix cause; Resume from checkpoint; Spark may fall back to standard
Batch failed **configuration**	Glue config chip → SME Republish → Resume
Vendor file **late / missed**	Chase vendor; Pull now or Waive
Change file **waiting** / **out of order**	Wait / Skip gap / Discard / SME apply with dual control
Snapshot truncation suspected	Confirm with vendor; resend or SME raise closes
Scheduler silent	Check Lambda / EventBridge; `python -m lbp.cli tick`

19. Web app UX patterns

React 18 + Vite; no UI framework. Pages hold lists; dialogs hold work on one item.

- Shared `Modal`: sizes sm/md/lg/xl; stacking; focus trap; Esc/backdrop closes top only; blocked while busy; ←/→ when not typing; full-screen on phone.
- `ConfirmModal`, `ReasonModal`, `useDialog()` for audited actions.
- Demo single-file build runs mock API in the browser.

20. Verified evidence & go-live checklist

Verified in-repo (summary)

- Functional pytest (mapping, rules, T1–T5, drift, replay, incremental/snapshot, vendors, sequencing, scheduler, API, Glue/Spark, lambdas/SFN).
- PostgreSQL 16 suite; Alembic migrations; concurrency (audit chain + sequencing lease).
- Crash safety / stale lease recovery; Cognito JWT + role matrix deny-by-default.
- NPI masking; hash-chained audit; ruff/hadolint; tofu validate; checkov.
- Throughput: see `docs/load-test-1m.txt`.

Required before go-live (PennyMac / Sonata)

1. Real SSE data dictionary and remaining Hangfire rule migration.
2. Live SSE intake integration (beyond typed staging).
3. SQL Server verification in CI.
4. Parallel/shadow run vs Hangfire; loan-by-loan reconcile.
5. Deploy into PennyMac AWS (ACM, domain, Cognito IdP, VPC/TGW).
6. Container build in CI with digest-pinned bases.
7. Security sign-off (threat model, pen test, InfoSec, data classification).
8. SR 11-7 model risk validation (live Bedrock).
9. DR: cross-region S3/RDS, RTO/RPO, tested restore.
10. Perf test vs real SSE; tune Glue concurrency / Spark workers / SSE throttle.
11. Per-vendor onboarding (load type, numbering, protected fields, due times, SFTP).
12. UAT, training, on-call handover.

21. Glossary

Term	Meaning
SSE	Servicing System of Engagement — target loan store
Feed	One vendor file layout + mapping + delivery profile
Batch	One registered file run through boarding
Publish	Sign and write the live config Glue will load
Dispatch	Start processing (SFN in AWS, thread locally)
Pre-flight	Schema/drift check before boarding
Sequencing gate	Enforces vendor file order
T1–T5	Exception risk tiers

Maker-checker	Two-person control for sensitive changes
NPI	Nonpublic personal information — masked by role

22. Appendix A — Personas in depth

Each demo persona maps to Cognito groups in production. Capabilities are enforced by `CAPS` / `canCap` (admin bypasses). For low-level design and code-level flows, see **Appendix B**.

Priya · Data Analyst

Roles: `analyst` · **Primary job:** Draft mappings & delivery profiles · **Guide default:** Setup a feed
Home screens: Vendors & Mapping, Rule Engine (author), How it works (Setup), Overview

Functions

- Own the first pass of vendor field mapping to SSE attributes
- Build and iterate delivery profiles (full / change / snapshot)
- Propose vendor & feed metadata without publishing live config
- Author draft rules in plain English for SME / Compliance to harden

How the platform helps

- Vendors & Mapping workbench with field inspector and value maps — no hand edits in SSE
- How it works → Setup path highlights her steps and deep-links to the right screens
- CAPS hide publish / approve / T5 actions so she cannot accidentally go live
- Audit-friendly drafts: every mapping change is versioned before SME sign-off

Dan · SME

Roles: `sme, analyst` · **Primary job:** Approve, publish, rules, vendor lifecycle · **Guide default:** Setup a feed
Home screens: Vendors & Mapping, Rule Engine, Import Monitor, Vendor Deliveries, Maintenance, Policy & Audit (read)

Functions

- Approve mappings and delivery profiles; publish signed feed config
- Move feeds through lifecycle; skip gaps / apply out-of-order with judgment
- Edit and peer-review boarding rules; shadow / canary / retire
- Resume blocked batches, override preflight with reason, read Maintenance
- Approve vendor schedules with Compliance when required

How the platform helps

- Single publish action writes HMAC-signed config Glue will load — no SSH / ticket sprawl

- Maintenance surfaces Glue / SFN / notebooks for ops truth without raw AWS console hunting
- Capability matrix + persona switch lets him rehearse Priya's drafts then approve as SME
- Rule Engine lifecycle (draft → review → deploy → monitor) keeps model-risk evidence with the rule

Maria · Boarding Ops

Roles: `ops` · **Primary job:** Upload, monitor batches, schedules, replay · **Guide default:** Run a file
Home screens: Import Monitor, Vendor Deliveries, How it works (Run), Overview, Analytics

Functions

- Upload or monitor vendor files; watch preflight and batch status
- Pause / resume / replay / discard batches within ops policy
- Maintain delivery schedules (SME/Compliance approve when gated)
- Skip sequencing gaps when authorized; escalate the rest

How the platform helps

- Import Monitor is the run cockpit — status, preflight, sequencing, actions in one place
- How it works → Run a file defaults for ops so training matches the day job
- Vendor Deliveries shows due windows and pull health without opening SFTP tooling
- Platform blocks actions outside her CAPS (no mapping publish, no T5, no policy edit)

Lee · Exception Reviewer

Roles: `reviewer` · **Primary job:** Work exception queues · **Guide default:** Run a file
Home screens: Exception Manager, How it works (Run), Overview, Analytics

Functions

- Clear T2–T3 exception queues so loans can continue boarding
- Apply suggested fixes, document decisions, escalate dual-control items
- Watch open counts and root-cause patterns day to day

How the platform helps

- Exception Manager queues by tier with root-cause rollups — not a raw Glue log dump
- Suggested fixes and confidence from rules / models cut average handle time
- Nav badge shows open exception count so backlog is visible without a report
- Guide marks "Your step" on exception handling; dual-control items route to Checker

Sam · Checker

Roles: `checker, reviewer` · **Primary job:** Checker approval on dual-control items · **Guide default:** Run a file
Home screens: Exception Manager, How it works (Run), Overview

Functions

- Second-person approve on T4 / maker-checker exception paths
- Can also work reviewer queues when capacity requires
- Refuse or return items that fail control standards

How the platform helps

- Platform enforces maker ≠ checker — same actor cannot self-approve
- Exception Manager surfaces pending checker work separately from first-pass review
- Audit trail records both maker and checker with reasons for examiners
- Same console as Lee so handoffs do not jump tools

Ana · Compliance

Roles: `compliance, checker` · **Primary job:** Regulated rules, policy, T5, audit · **Guide default:** Setup a feed
Home screens: Policy & Audit, Exception Manager (T5), Rule Engine, Vendor Deliveries (approve), Overview

Functions

- Own T5 (OFAC / fraud halt) and compliance-gated exception acts
- Edit exception / auto-apply policy; kill-switch T1 when override rates spike
- Approve regulated rules and vendor lifecycle / schedule gates
- Read full audit history; confirm NPI masking for other roles

How the platform helps

- Policy & Audit is purpose-built — thresholds, kill switch, demo scale, evidence trail
- CAPS reserve complianceAct / editPolicy so ops and analysts cannot bypass regulation
- Rule Engine requires compliance sign-off on regulatory-tagged rules before go-live
- Same checker powers when dual-control needs a compliance-grade second signature

Platform Admin

Roles: `admin (+ all)` · **Primary job:** Full access + demo setup + Maintenance · **Guide default:** Setup a feed
Home screens: All pages including Maintenance and demo controls on Policy & Audit / Overview

Functions

- Bypass role checks for break-glass and environment setup
- Run demo seed / scale tooling; validate every screen

- Read Maintenance artifacts (Glue, orchestration, notebooks)
- Support production cutover and training with persona switching in demo mode

How the platform helps

- Admin role short-circuits CAPS for recovery without redeploying IAM mid-incident
- Demo setup recreates realistic volume for UAT and executive walkthroughs
- Maintenance + full nav visibility for platform owners validating AWS wiring
- Persona switcher still available in demo so training can role-play without new accounts

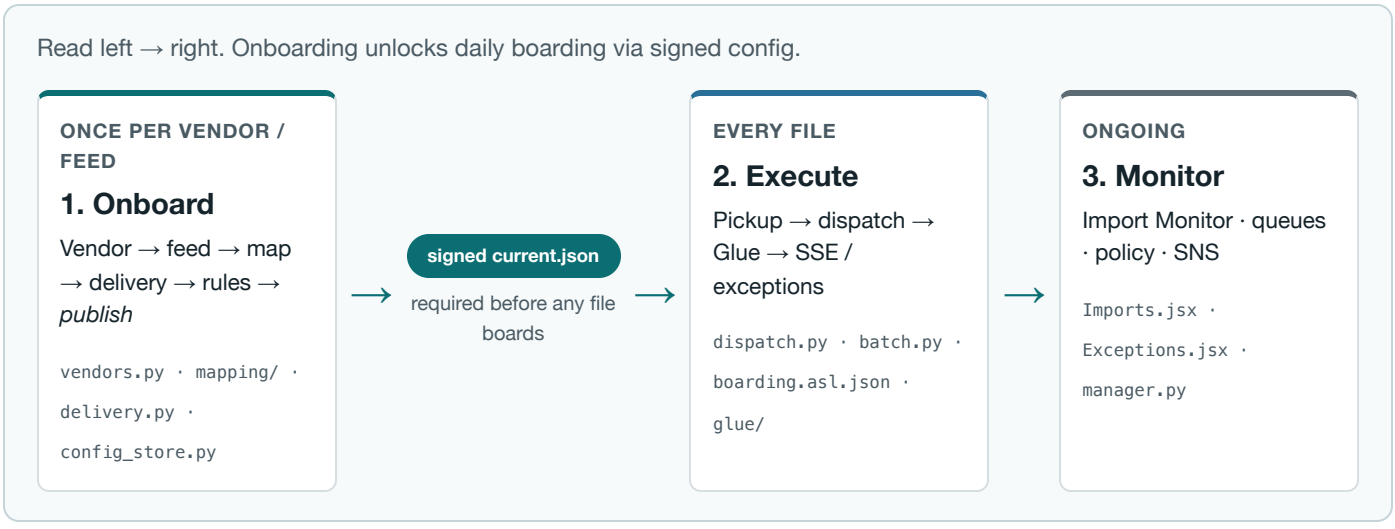
---

23. Appendix B — LLD: onboarding, daily execution & monitoring

Low-level design starts from the platform lifecycle, then drills into **vendor onboarding** (once per feed) and **daily execution & monitoring** (every file). Each flow is a single print board (kept on one page) with the code file on every stage. Control plane configures and dispatches; Glue (or the local worker) is the only place loan bytes are transformed.

B.1 High-level flow

One spine: onboard once, then every file executes, then people monitor. Signed `current.json` is the bridge.

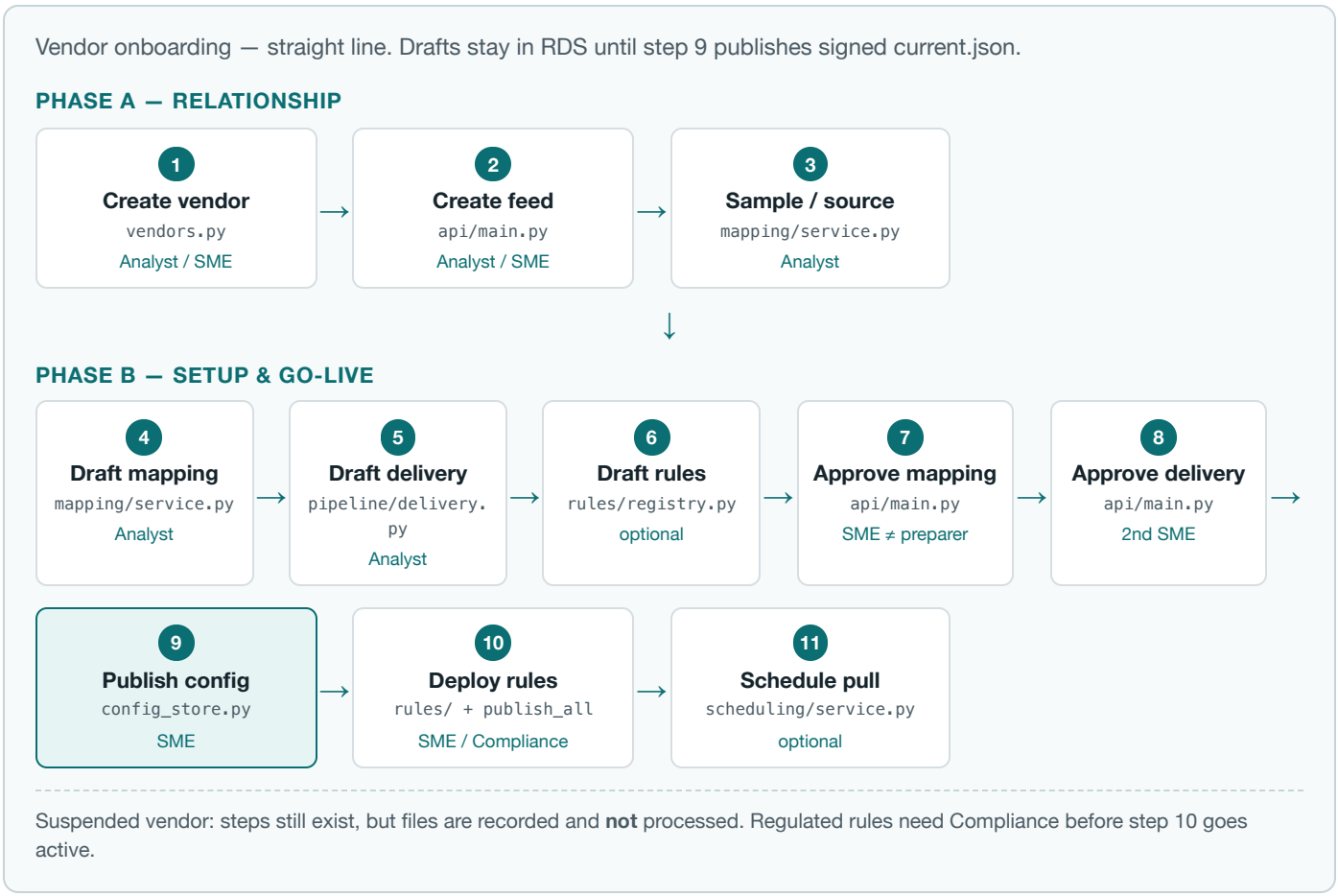


Track	Ends when	Primary code
Vendor onboarding	`current.json` signed for the feed	`backend/lbp/vendors.py`, `mapping/service.py`, `pipeline/delivery.py`, `config_store.py`, `rules/registry.py`
Daily execution	Batch outcome + SNS; loans in SSE or exception queues	`dispatch.py`, `pipeline/batch.py`, `aws/stepfunctions/boarding.asl.json`, `aws/glue/*.py`

Monitoring	Humans clear blocks / exceptions; audit intact	`frontend/src/pages/Imports.jsx`, `Exceptions.jsx`, `Governance.jsx`, `exceptions/manager.py`

B.2 Vendor onboarding — all steps

Phases A–B. Drafts stay in RDS until SME approve + publish. Suspended vendors still accept file records but do not board.



Step list

1. **Create vendor** — identity, type, status (active / suspended).
2. **Create feed** — one file layout per feed under that vendor.
3. **Register / sample** — upload a sample or create source so mapping can start.
4. **Draft mapping** — analyst maps vendor columns → SSE attributes (versioned drafts).
5. **Draft delivery profile** — full / incremental / snapshot, sequencing, protected fields.
6. **Author rules (optional)** — plain-English / DSL drafts; regulated tags flagged.
7. **SME approve mapping** — approver ≠ preparer; triggers publish for that feed.
8. **SME approve delivery** — second SME; publish again with delivery in the bundle.
9. **Deploy rules** — peer / Compliance as required → active|canary|shadow → `publish_all`.
10. **Confirm Glue config vN** — console chip; in sync with active mapping.

11. **Optional schedule** — Ops proposes pull window; SME/Compliance approve; hosts allow-listed.

Code map — onboarding

Step	What happens	Code file	Entry
1. Create vendor	Persist vendor + audit lifecycle	`backend/lbp/vendors.py`	`create_vendor()`
1–2. API surface	HTTP routes for vendors / feeds / sources	`backend/lbp/api/main.py`	`create_source(), vendor routes`
2–3. Feed / sample	Register source, parse sample headers	`backend/lbp/mapping/service.py`	`register_source()`
3. Inspect sample	Field inspector / value maps	`backend/lbp/mapping/inspect.py`	`inspect helpers`
3. Parse options	File format / delimiter / sheet	`backend/lbp/ingestion/parser.py`	`read_sample(), ParseOptions`
4. Mapping drafts	Versioned mapping CRUD + AI assist	`backend/lbp/mapping/service.py`	`mapping draft / revise`
4. SSE dictionary	Target attribute catalog	`backend/lbp/schema/sse_target.py`	`SSE field defs`
5. Delivery profile	Propose / decide load type & sequencing	`backend/lbp/pipeline/delivery.py`	`propose_profile(), decide_profile()`
5. Approve delivery	Second SME gate → publish	`backend/lbp/api/main.py`	`approve_delivery()`
6–9. Rules	Author, transition, registry	`backend/lbp/rules/registry.py`	`RuleRegistry / transitions`
6. Rule execute shape	JSON Logic executor used later in Glue	`backend/lbp/rules/executor.py`	`RuleExecutor`
7. Approve mapping	SME ≠ preparer → publish	`backend/lbp/api/main.py`	`approve_mapping()`
7–9. Signed publish	build + HMAC + current.json	`backend/lbp/config_store.py`	`publish(), publish_all(), load()`
2. Move feed	Re-home feed → publish_all	`backend/lbp/vendors.py`	`move_feed()`
11. Schedules	Cron windows, pull-now	`backend/lbp/scheduling/service.py`	`tick(), pull_now()`
11. Pullers	SFTP / S3 fetchers	`backend/lbp/scheduling/pullers.py`	`pull implementations`
11. Cron parse	Schedule expressions	`backend/lbp/scheduling/cron.py`	`cron helpers`

Caps / roles	UI capability gates	`frontend/src/roles.js`	`can(), canCap(), CAPS`
Console — vendors	Vendors & Mapping UI	`frontend/src/pages/Sources.jsx`	`Workbench / mapping`
Console — delivery	Delivery profile dialogs	`frontend/src/pages/DeliveryProfile.jsx`	`edit / review`
Console — vendors list	Vendor cards / feeds	`frontend/src/pages/Vendors.jsx`	`VendorWorkspace`
Console — rules	Rule Engine UI	`frontend/src/pages/Rules.jsx`	`AuthorModal / transitions`
Console — schedules	Vendor Deliveries UI	`frontend/src/pages/Deliveries.jsx`	`ScheduleEditor`
Deploy backfill	Missing bundles on deploy	`aws/lambdas/handlers.py`	`migrate()`

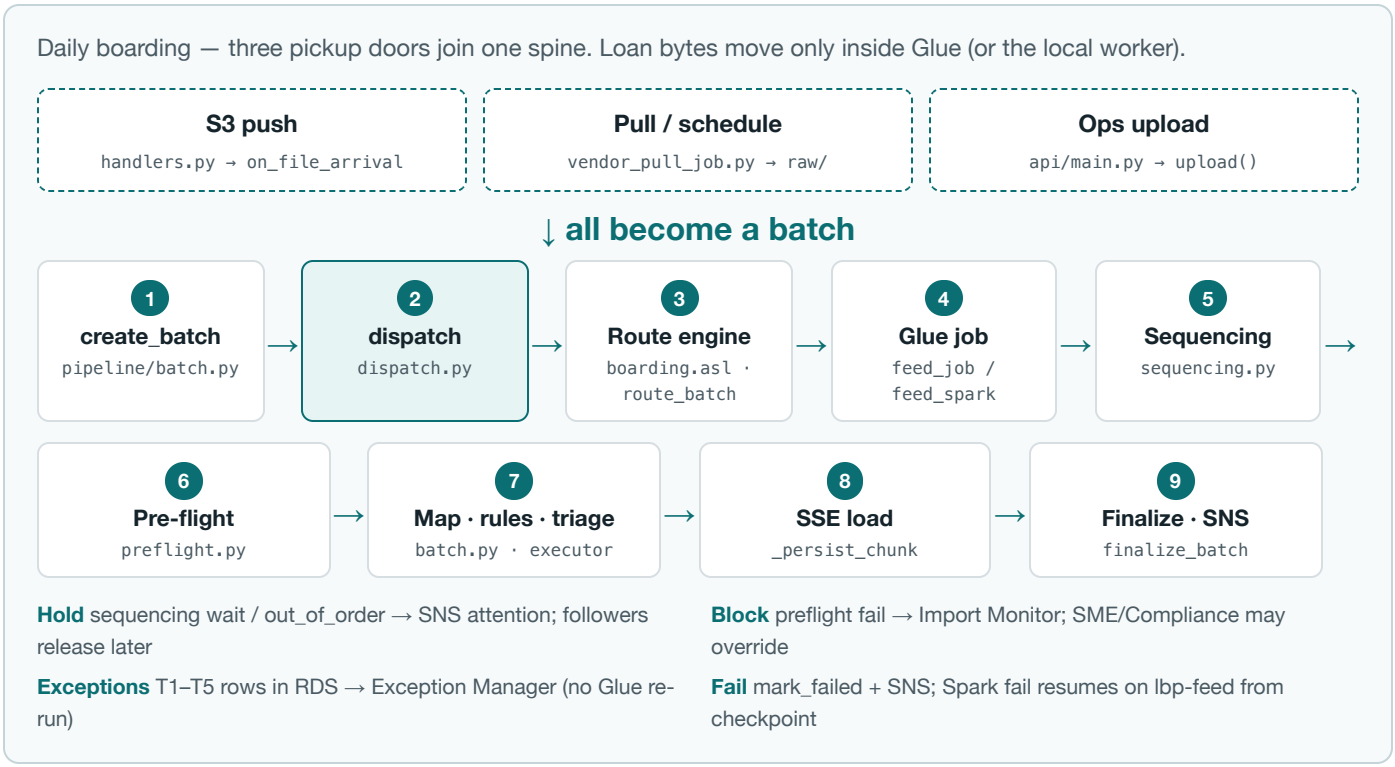
Publish contract (snippet)

```
# backend/lbp/config_store.py — publish()
body = build(source_id)                 # mapping + delivery + live rules
sha, sig = _sign(body)                  # Secrets Manager HMAC key
_write(source_id, f"v{version}.json", data)
_write(source_id, "current.json", data) # only pointer Glue trusts
audit(actor, "config.publish", …)

# Glue / local worker — same wheel
cfg = config_store.load(source_id)      # verify or batch.config_rejected
```

B.3 Daily execution

Phases C–E. Same boarding path for S3 push, scheduled pull, and Ops upload. API Lambda never transforms loan rows.



Step list

1. **Pickup** — S3 ObjectCreated, vendor-pull into raw/, or Ops upload.
2. **Register batch** — `create_batch` with path, feed, load type; dedupe linked batches.
3. **Dispatch** — local thread or Step Functions `boarding.asl.json`.
4. **Route** — choose `lbp-feed` vs `lbp-feed-spark`.
5. **Load config** — signature-verify `current.json`.
6. **Sequencing gate** — wait / out_of_order / proceed.
7. **Pre-flight** — schema/drift; block → Import Monitor; override path for SME/Compliance.
8. **Board** — parse → map → RuleExecutor → Triage T1–T5 → apply (full / change / snapshot).
9. **Persist** — throttled micro-batches to SSE; checkpoint; reconcile; advance cursor.
10. **Finalize** — outcome, SNS, release queued followers via `dispatch` again.
11. **Failures** — `mark_failed`; Spark failure resumes on standard engine from checkpoint.

Code map — execution

Step	What happens	Code file	Entry
1. S3 / EventBridge arrival	Register file from object event	`aws/lambdas/handlers.py`	`on_file_arrival()`
1. Scheduler tick	Due windows → pull jobs	`aws/lambdas/handlers.py`	`scheduler_tick()`
1. Vendor pull job	SFTP/S3 → raw/incoming	`aws/glue/vendor_pull_job.py`	`Glue job entry`

1. Pull orchestration	start_glue vendor-pull	`backend/lbp/scheduling/service.py`	`pull_now(), tick()`
1. Ops upload	Multipart upload → dispatch	`backend/lbp/api/main.py`	`upload()`
2. Create batch	Persist batch row + path	`backend/lbp/pipeline/batch.py`	`create_batch()`
3. Dispatch	Local thread or SFN start	`backend/lbp/dispatch.py`	`dispatch(), start_glue()`
3. Boarding ASL	Route → Glue.sync → Finalize → SNS	`aws/stepfunctions/boarding.asl.json`	`State machine`
4. Route engine	standard vs spark choice	`aws/lambdas/handlers.py`	`route_batch()`
4–9. Standard Glue	Driver job wrapper	`aws/glue/feed_job.py`	`invokes lbp pipeline`
4–9. Spark Glue	mapInPandas + fallback	`aws/glue/feed_spark_job.py`	`Spark entry`
5–9. Pipeline core	Same code local + Glue	`backend/lbp/pipeline/batch.py`	`run_batch(), _persist_chunk()`
5. Signed config load	Verify HMAC bundle	`backend/lbp/config_store.py`	`load()`
6. Sequencing	Vendor file order gate	`backend/lbp/pipeline/sequencing.py`	`check / wait / release`
7. Pre-flight	Schema / drift checks	`backend/lbp/pipeline/preflight.py`	`check_file()`
7. Override preflight	SME/Compliance with reason	`backend/lbp/api/main.py`	`override_preflight()`
8. Parse rows	Chunked file iteration	`backend/lbp/ingestion/parser.py`	`iter_rows()`
8. Rules	Per-loan validation	`backend/lbp/rules/executor.py`	`RuleExecutor`
8. Triage tiers	T1–T5 classification	`backend/lbp/exceptions/manager.py`	`Triage`
8. Change / snapshot	Incremental apply plans	`backend/lbp/pipeline/changes.py`	`change apply`
8. Delivery decide	Load-type behaviour	`backend/lbp/pipeline/delivery.py`	`profile_of(), display_sequence()`
9. Glue job catalog	Job names / concurrency knobs	`backend/lbp/glue_jobs.py`	`job helpers`
10. Finalize	Outcome + release followers	`aws/lambdas/handlers.py`	`finalize_batch()`
11. Mark failed	Terminal failure + notify	`aws/lambdas/handlers.py`	`mark_failed()`
Replay held loans	Glue --mode replay	`backend/lbp/pipeline/replay.py`	`replay_batch()`
Storage paths	Local/S3 path helpers	`backend/lbp/storage.py`	`local_path()`

Dispatch (snippet)

```
# backend/lbp/dispatch.py
def dispatch(batch_id: str) -> str:
    if not aws_mode():
        start_batch_async(batch_id)   # pipeline.batch — same as Glue
        return "local"
    return sfn.start_execution(
        stateMachineArn=…, name=f"{batch_id}-…",
        input={"batch_id", "source_id", "bucket", "key"})
```

B.4 Daily monitoring & human follow-through

After Glue writes outcomes and exception rows, people work only the judgment paths. Resolving an exception does not re-run Glue unless Ops explicitly replays a failed batch.

Daily monitoring — three consoles. Ops re-enters dispatch; reviewers finalize loans in the DB.

Batches
`Imports.jsx · api/main.py`
- Status · preflight · sequencing
- pause / resume / skip-gap
- replay → dispatch again

Exceptions
`Exceptions.jsx · manager.py`
- T2–T3 reviewer
- T4 maker → checker
- T5 compliance only
- → _finalize_loan

Governance
`Governance.jsx · analytics/`
- T1 kill switch · policy
- Audit chain
- Overview KPIs · SNS
- Maintenance (Glue / SFN)

Step list

1. **Watch batches** — Import Monitor status, preflight blocks, sequencing waits, SNS.
2. **Ops actions** — pause / resume / discard / skip-gap / replay (re-enters `dispatch`).
3. **Clear T2–T3** — Exception Manager review + suggested fixes.
4. **T4 maker-checker** — second person ≠ maker; then finalize loan.
5. **T5 compliance** — OFAC/fraud halt; compliance-only act.
6. **Policy knobs** — T1 kill switch and thresholds (DB-live on next row).
7. **Audit / analytics** — hash-chained audit; Overview KPIs; Maintenance for Glue/SFN artifacts.

Code map — monitoring

Step	What happens	Code file	Entry
1. Batch list / detail	Import Monitor API	`backend/lbp/api/main.py`	`list_batches(), pause/resume`
1. Console — imports	Ops cockpit UI	`frontend/src/pages/Imports.jsx`	`Imports / Preflight / UploadModal`
2. Pause / resume / override	Control-plane batch controls	`backend/lbp/api/main.py`	`pause_batch, resume_batch, override_preflight`

2. Replay	Held-loan replay path	`backend/lbp/pipeline/replay.py`	`replay_batch()`
2. Re-dispatch	After ops action	`backend/lbp/dispatch.py`	`dispatch()`
3–5. Exception API	Queue + resolve	`backend/lbp/api/main.py`	`list_exceptions`, `resolve_exception`
3–5. Exception core	Actions + finalize loan	`backend/lbp/exceptions/manager.py`	`ExceptionManager`, `_finalize_loan()`
3. Console — exceptions	Queues / tiers UI	`frontend/src/pages/Exceptions.jsx`	`ExceptionModal, RootCauses`
6. Policy	Thresholds + kill switch	`frontend/src/pages/Governance.jsx`	`PolicyModal`
6–7. Audit reads	Audit API	`backend/lbp/api/main.py`	`audit list / verify`
7. Analytics	KPI services	`backend/lbp/analytics/service.py`	`analytics aggregates`
7. Console — overview	Executive KPIs	`frontend/src/pages/Overview.jsx`	`Overview`
7. Console — analytics	Charts	`frontend/src/pages/Analytics.jsx`	`Analytics`
7. Maintenance	Glue / SFN / notebook artifacts	`frontend/src/pages/Maintenance.jsx`	`Maintenance`
Caps	Who can click what	`frontend/src/roles.js`	`CAPS / canCap`
SNS / metrics	Boarding ASL notify + Glue metrics	`aws/stepfunctions/boarding.asl.json`	`Notify* states`

> **Remember.** Onboarding ends at signed `publish`. Daily execution starts at `dispatch`. Monitoring clears human gates — loan transformation stays in Glue.

End of guide · Loan Boarding Console · 2026-11-04